

Implementación de protocolo actualizable de firma digital basado en retículos



Universitat Oberta
de Catalunya

Juan José Campos Rosa

Protocolos criptográficos y aplicaciones de seguridad

Máster en Ciberseguridad y Privacidad

Nombre del director/a de TF:

Ramsès Fernàndez i València

Nombre del/de la PRA:

Pau Perea Paños

2 de junio de 2026



Esta obra está sujeta a una licencia de Reconocimiento

<https://creativecommons.org/licenses/by-nc/3.0/es/>

Ficha del Trabajo Final

Título del trabajo:	Implementación de protocolo actualizable de firma digital basado en retículos
Nombre del autor/a:	Juan José Campos Rosa
Nombre del director/a de TF:	Ramsès Fernàndez i València
Nombre del/de la PRA:	Pau Perea Paños
Fecha de entrega:	2 de junio de 2026
Titulación o programa:	Máster en Ciberseguridad y Privacidad
Área del trabajo final:	Protocolos criptográficos y aplicaciones de seguridad
Idioma del trabajo:	Castellano
Palabras clave:	firmas actualizables, retículos, GPV, CRSST21, convolución gaussiana

Resumen del trabajo

En este trabajo se explica, implementa y analiza una propuesta de esquema de firma digital actualizable basada en retículos, planteada como una evolución del marco CRSST21. Para ello, se revisan los conceptos matemáticos y criptográficos necesarios para contextualizar el trabajo. A continuación, se justifican las mejoras que introduce el nuevo esquema y se presenta una descripción detallada de su funcionamiento. A esto le sigue la implementación de una prueba de concepto en Python, orientada a reproducir experimentalmente algunos aspectos funcionales y estadísticos del esquema mediante simulaciones controladas. Finalmente, se analizan los resultados de dichas simulaciones, a fin de comprobar empíricamente las propiedades esperadas del esquema. Como resultado, en las condiciones simuladas, se observa que el prototipo reproduce el efecto esperado de enmascaramiento probabilístico la acción del token de actualización sobre la distribución de las firmas. No obstante, también se identifica una limitación relevante: el esquema hereda el problema de crecimiento elevado de los parámetros de dispersión y de las cotas de aceptación.

Abstract

This work explains, implements and analyses a proposal for an updatable digital signature scheme based on lattices, presented as an evolution of the CRSST21 framework. To this end, the mathematical and cryptographic concepts required to contextualise the work are first reviewed. The improvements introduced by the new scheme are then justified, before presenting a detailed description of its operation. This is followed by the implementation of a proof of concept in Python, aimed at experimentally reproducing some functional and statistical aspects of the scheme through controlled simulations. Finally, the results of these simulations are analysed in order to empirically assess the expected properties of the scheme. As a result, under the simulated conditions, the prototype is observed to reproduce the expected probabilistic masking effect of the update token on the distribution of signatures. However, a relevant limitation is also identified: the scheme inherits the problem of high growth in the dispersion parameters and acceptance bounds.

Índice general

Introducción	9
Estado del arte	10
Metodología	12
1. Definiciones generales	13
1.1. Conceptos previos	13
1.2. CRSST21	17
1.3. Análisis de seguridad	19
2. Descripción del esquema propuesto	26
2.1. Análisis conceptual de las mejoras introducidas	26
2.2. Descripción del nuevo esquema	28
2.3. Resultados de seguridad	29
3. Implementación del esquema	31
3.1. Algoritmos adaptados para la implementación	31
3.1.1. TrapGen	31
3.1.2. SamplePre	32
3.2. Código del esquema	36
4. Evaluación y análisis de resultados	62
4.1. Ajuste de parámetros	62
4.1.1. Motivación	62
4.1.2. Cálculo de parámetros	65
4.2. Comparación de rendimiento entre esquemas	68
4.3. Análisis estadístico	70
Conclusión	72
Bibliografía	74
Apéndice	75

A.	Esquema VE impostor	75
B.	Simulador y monitorización	76
C.	Análisis estadísticos de la dispersión efectiva	80
D.	Análisis estadístico PCA	83

Índice de figuras

1.1. Retículo generado por B	14
1.2. Retículo generado por B'	15
1.3. Ejemplos de información obtenida directamente (cuadros verdes) e información inferida (cuadros azules). Tabla obtenida de [1].	21
1.4. Esquema de $Exp_{US,A}^{us-uu-cma}(\lambda, N)$ extraído de [8].	22
1.5. Esquema de $Exp_{US,A}^{us-euf-cma}(\lambda, N)$ extraído de [8]	23
2.1. Ejemplo de corrección X mediante suma de ruido gaussiano.	28
4.1. Evolución de s_e y β_e para $e = 1, \dots, 5$	66
4.2. Crecimiento de s_e por épocas respecto p_0 en escala logarítmica.	67
4.3. Crecimiento de s_e por épocas con $p_0 = 0,3$ en escala logarítmica.	67
4.4. Análisis PCA sobre dos muestras de tamaño 5000 generadas con CRSST21 y USGPV.	71

Índice de tablas

4.1. Estimaciones del coste de búsqueda para recuperar R y del valor de la norma espectral en función de p_0	64
4.2. Comparación de los parámetros utilizados en USGPV y CRSST21 para la simulación.	68
4.3. Comparación del uso de recursos y tiempos de ejecución entre USGPV y CRSST21.	69
4.4. Análisis de la época 1 incluyendo tiempo de uso efectivo de CPU.	70
4.5. Análisis del error entre covarianza esperada y calculada por tamaño de la muestra.	70

Introducción

Cuando hablamos de firma digital, resulta evidente su importancia en un entorno cada vez más digitalizado. La posibilidad de verificar a distancia que un archivo transmitido por canales inseguros no ha sido modificado y que, además, cuenta con la aprobación de su emisor o autor, tiene un valor fundamental en un contexto marcado por el crecimiento de la información digital y de los trámites en línea.

Sin embargo, en la práctica los pares de claves asociados a la firma pueden cambiar con el tiempo por diferentes motivos, ya sea por buenas prácticas de seguridad, extravío o porque la clave privada ha sido comprometida (entre otros motivos). Este hecho introduce un problema operativo: los documentos firmados con la firma antigua quedan inutilizables y requieren ser refirmados. Para enfrentar este problema existen varias familias de soluciones. La más inmediata sería solicitar al usuario que firme de nuevo todos los documentos asociados a la clave antigua, pero esta opción resulta tediosa y poco escalable. Como alternativa, surgen los esquemas de firma digital actualizables, que permiten adaptar firmas ya emitidas a nuevas claves sin necesidad de repetir manualmente el proceso para cada documento.

En el contexto actual de la criptografía post-cuántica, han comenzado a aparecer propuestas de firma digital actualizable basadas en retículos. Esto hace especialmente relevante su estudio, así como su implementación y evaluación mediante simulaciones controladas cercanas a un escenario práctico. El objetivo general de este trabajo es evaluar el protocolo de firma digital actualizable basado en retículos, propuesto en el artículo “*Updatable Signature from Lattices*”, publicado en septiembre de 2025 de la mano de Haotian Yin, Jie Zhang, Wanxin Li, Yuji Dong, Eng Gee Lim y Dominik Wojtczak. Presentaremos un análisis del rendimiento del esquema mediante simulaciones controladas.

Estado del arte

El uso de retículos en criptografía moderna comienza a consolidarse a mediados de los años noventa, especialmente a partir del trabajo de Ajtai, donde se muestra que ciertos problemas de retículos pueden utilizarse para construir primitivas criptográficas seguras. Poco después aparecen propuestas como GGH, que introducen la idea de utilizar la dificultad de resolver problemas geométricos sobre retículos como base para construir esquemas de clave pública.

De manera intuitiva, un retículo es un conjunto de puntos distribuidos de forma regular en el espacio. Más formalmente, puede verse como el conjunto de combinaciones lineales enteras de una base de vectores. Una característica importante es que un mismo retículo puede estar generado por bases distintas, algunas más “buenas” que otras. Esta diferencia entre bases es una de las ideas que hace útiles a los retículos en criptografía: ciertos problemas pueden ser difíciles si solo se conoce una base arbitraria, pero tratables si se dispone de una base con buena geometría, vectores cortos y Gram-Schmidt controlado.

En este contexto aparecen varios problemas computacionalmente difíciles, como SVP, CVP y SIS. El problema SIS (*Small Integer Solution*) fue introducido por Ajtai en 1996 y consiste, de forma simplificada, en encontrar un vector entero no nulo y suficientemente corto que verifique una ecuación modular lineal.

A partir de esta base, un avance especialmente relevante fue el esquema de Gentry, Peikert y Vaikuntanathan, conocido como GPV, publicado en 2008. Este marco introduce una construcción de firma tipo *hash-and-sign* basada en retículos. La idea principal es que la clave pública define una función lineal modular difícil de invertir, mientras que la clave secreta contiene una *trapdoor* que permite encontrar preimágenes cortas. Así, para firmar un mensaje, se calcula un valor asociado al mensaje y después se utiliza la *trapdoor* para muestrear una preimagen corta de ese valor. Esta preimagen es la firma. La parte delicada es que dicha preimagen no debe elegirse de cualquier forma, sino siguiendo una distribución adecuada para no filtrar información sobre la clave secreta.

Esta idea sirve de base al marco CRSST21 que aparece en 2021 para adaptar GPV al contexto de firmas actualizables. En CRSST21, el sistema se divide en épocas y, en cada cambio de época, se genera un nuevo par de claves junto con un token de actualización. El objetivo es que una firma válida en una época pueda actualizarse para seguir siendo válida bajo las claves de la época siguiente. Por tanto, mientras que GPV proporciona el mecanismo básico para firmar mediante preimágenes cortas, CRSST21 añade una estructura temporal que permite que las firmas evolucionen junto con las claves.

Sin embargo, este proceso introduce nuevos problemas de seguridad. En particular, una actualización determinista puede filtrar información sobre el token de actualización o permitir distinguir entre firmas frescas y firmas actualizadas. Precisamente por eso, surge la necesidad de refinar el proceso de actualización para blindarlo probabilísticamente. A finales de 2025 se publicó el paper “*Updatable Signature from Lattices*” que aquí tratamos, en el que se busca solucionar este problema.

Metodología

Este trabajo sigue una metodología teórico-práctica. Por un lado, se estudian los fundamentos matemáticos y criptográficos necesarios para comprender el esquema seleccionado y, por otro, se desarrolla un prototipo demostrativo que permita analizar su funcionamiento y su utilidad en la práctica.

En una primera fase, se realizará una revisión bibliográfica sobre criptografía post-cuántica, criptografía basada en retículos y sobre el esquema concreto que se va a estudiar. Esta parte permitirá entender el contexto general del problema e introducirnos de manera consistente en el tema.

Se desarrollará la base teórica necesaria para el trabajo sin entrar en una exposición exhaustiva de toda la teoría de retículos. El foco estará en aquellos conceptos, propiedades y problemas matemáticos que sean realmente necesarios para entender el protocolo y justificar su seguridad.

En una segunda fase, se llevará a cabo la descripción técnica del nuevo esquema propuesto en el artículo [8]. En esta fase se explicará su funcionamiento, las primitivas que emplea, los supuestos de seguridad sobre los que se apoya y sus principales ventajas y limitaciones.

Se procederá a la implementación de un prototipo demostrativo en Python. El objetivo de esta parte no será desarrollar una herramienta lista para producción, sino una implementación clara y funcional que permita trasladar la propuesta teórica a casos prácticos, comprobar su comportamiento y mostrar de forma comprensible cómo opera el esquema.

En una tercera fase, la simulación y la evaluación del protocolo se realizarán también en Python. Para ello, se diseñarán experimentos controlados en los que pueda apreciarse la utilidad del nuevo esquema propuesto. La implementación desarrollada debe entenderse como una prueba de concepto académica. No pretende proporcionar una implementación criptográficamente segura para una puesta en producción.

A modo de resumen, la metodología planteada se resume en una revisión bibliográfica para establecer un marco teórico consistente y robusto sobre el que posteriormente se explicará el esquema en cuestión. Es justo después cuando se desarrollará un prototipo demostrativo para poner a prueba el protocolo y ejecutar así un análisis del mismo a partir de las métricas recolectadas durante las simulaciones.

Capítulo 1

Definiciones generales

1.1. Conceptos previos

Comenzaremos repasando algunos conceptos clave en la criptografía basada en retículos. Como es natural, hay que comenzar dando una definición formal de lo que es un retículo y algunas de sus propiedades más útiles para nuestro caso.

Definición 1.1. Un retículo n -dimensional $\mathcal{L}$ es cualquier subgrupo aditivo de $\mathbb{R}^n$ que sea discreto, es decir, $\forall x \in \mathcal{L}$ existe un entorno donde x es el único punto del retículo.

Recordamos que ser un subgrupo requiere que $\mathbf{0} \in \mathcal{L}$, y $-x, x + y \in \mathcal{L} \ \forall x, y \in \mathcal{L}$. En general, un retículo es un conjunto de combinaciones lineales enteras de un conjunto de vectores a los que llamamos base. No obstante, es importante recalcar que podemos definir un retículo n -dimensional de dos formas:

- Podemos dar una base del retículo B , lo que implica que el retículo $\mathcal{L}(B)$ sería el conjunto:

$$\mathcal{L}(B) = B \cdot \mathbb{Z}^n = \left\{ \sum_{i=1}^n z_i \cdot b_i : z_i \in \mathbb{Z} \right\}$$

Llamaremos rango de $\mathcal{L}$ al valor m y dimensión de $\mathcal{L}$ al valor n . Cuando $m = n$ diremos que es un retículo de rango máximo. Si pensamos en B como una matriz $n \times m$

$$B = \begin{pmatrix} b_{1,1} & \cdots & b_{m,1} \\ \vdots & \ddots & \vdots \\ b_{1,n} & \cdots & b_{m,n} \end{pmatrix}$$

con las columnas formadas por los vectores de la base $B = \{b_1, \dots, b_m\}$, entonces podemos escribir

$$\mathcal{L} := \mathcal{L}(B) = \{B \cdot z \mid z \in \mathbb{Z}^m\} = B \cdot \mathbb{Z}^m.$$

Dada una base B de un retículo $\mathcal{L}$, definimos

$$\text{span}(\mathcal{L}) := \text{span}(B) = \{B \cdot y \mid y \in \mathbb{R}^m\}$$

- Pero también podemos definir un retículo usando condiciones como que verifiquen cierta ecuación:

$$\mathcal{L} = \{z \in \mathbb{Z}^n : Az = 0, A \in \mathbb{Z}^{m \times n}\}$$

Es inmediato comprobar que es un subgrupo aditivo (por linealidad) y que es discreto.

Lema 1.2. Dadas dos bases $B = \{b_1, \dots, b_m\}$ y $B' = \{b'_1, \dots, b'_m\}$ de $\mathbb{R}^n$ que generan un retículo $\mathcal{L}$, existe una matriz unimodular (es decir, invertible en $\mathcal{M}^{m \times m}(\mathbb{Z})$) tal que $B = B' \cdot U$. En este caso, diremos que las bases son equivalentes.

Este resultado, lo hemos obtenido de [2]. Una consecuencia inmediata es que si dos bases son equivalentes, generan el mismo retículo. No obstante, es inmediato comprobar que hay bases mejores que otras.

Ejemplo 1.3. Consideremos el retículo generado por $B = \{(2, 0, 0), (0, 2, 0), (0, 0, 2)\}$.

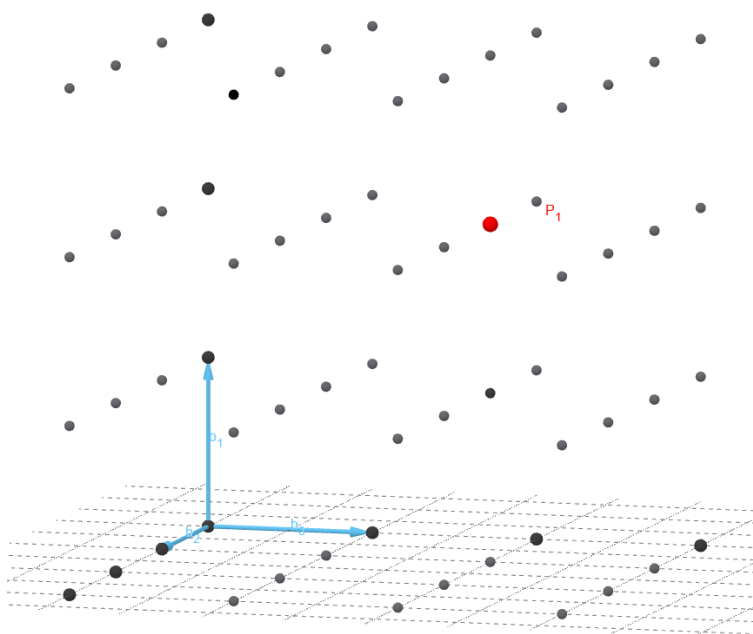


Figura 1.1: Retículo generado por B .

Ahora, en virtud del lema 1.2, la base $B' = \{(4, 6, 2), (2, 4, 2), (2, 2, 2)\}$ genera el mismo retículo ya que $(B')^{-1} \cdot B = U$ con $\det(U) = 1$.

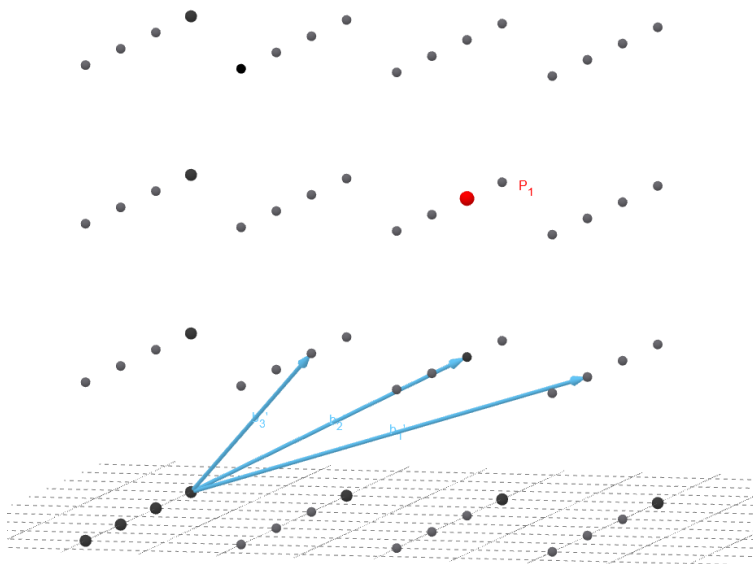


Figura 1.2: Retículo generado por B' .

Parece razonable pensar que llegar al punto P_1 es más sencillo usando la primera base que la segunda, pese a generar el mismo retículo.

Esta idea es la que está detrás de muchos de los esquemas basados en retículos. Una forma de saber si una base es buena o no es fijarnos en su ortogonalización por Gram-Schmidt.

Definición 1.4 (Ortogonalización Gram-Schmidt). Dado una base $B = \{b_1, \dots, b_n\}$, podemos calcular otra base $B^* = \{b_1^*, \dots, b_n^*\}$ tal que sus elementos son ortogonales entre sí. Los elementos de B^* se calculan de la siguiente forma:

$$b_1^* = b_1 \quad ; \quad b_i^* = b_i - \sum_{j < i} \frac{\langle b_i, b_j^* \rangle}{\langle b_j^*, b_j^* \rangle} b_j^*, \quad i \geq 2$$

Hay que aclarar que cuando se habla de que la base ortogonalizada es otra base, nos referimos a una base del espacio $\mathbb{R}^m$. De hecho, en general los vectores ortogonalizados no solo no generan el retículo, sino que ni siquiera están en el retículo generado por la base original. Siguiendo con la calidad de la base, decimos que una base es buena cuando tiene Gram-Schmidt controlado, i.e, que $\max_i \|b_i^*\|$ esté razonablemente acotado. Esto refleja que la geometría interna de la base es buena, ortogonal y sencilla. Los cálculos se simplifican y aplicar ciertos algoritmos se vuelve más eficiente.

Por otro lado, existen numerosos problemas complicados de resolver computacionalmente, lo cual nos motiva a buscar aplicaciones criptográficas. Mencionamos algunos de estos problemas.

Definición 1.5 (Shortest Vector Problem (SVP)). Dada una base arbitraria B de un retículo $\mathcal{L}(B)$, encontrar un vector no nulo tal que su norma sea igual a la distancia mínima del

retículo, i.e, un vector v_0 tal que $\|v_0\| = \min_{v \in \mathcal{L} \setminus \{0\}} \|v\|$.

Definición 1.6 (Closest Vector Problem (CVP)). Dada una base arbitraria B de un retículo $\mathcal{L}(B)$ n -dimensional y un vector $t \in \mathbb{R}^n$, encontrar $v_0 \in \mathcal{L}(B)$ tal que $\|v_0 - t\| = \min_{v \in \mathcal{L}} \|v - t\|$

Definición 1.7 (Small Integer Solution (SIS)). Dado un entero q , una matriz $A \in \mathbb{Z}^{n \times m}$ y un número real β , encontrar un vector $v \in \mathbb{Z}^m$ tal que $Av \equiv 0 \pmod{q}$ y que $\|v\| \leq \beta$.

También podemos definir su versión no homogénea (ISIS) que aparecerá igualmente más adelante. A continuación, recordamos notación importante de teoría de complejidad.

Definición 1.8. Sean $f(x), g(x)$ dos funciones cualesquiera y K una constante. Diremos que $f(x) = \mathcal{O}(g(x))$ si $\exists x_0$ tal que $\forall x \geq x_0$ $f(x) \leq K \cdot g(x)$. Por otro lado, diremos que $f(x) = \omega(g(x))$ si

$$\lim_{x \rightarrow +\infty} \frac{f(x)}{g(x)} = +\infty$$

Para terminar con esta sección, vamos a presentar una definición formal de esquema de firma actualizable [1].

Definición 1.9 (Updatable Signature (US)). Un esquema de firma actualizable US sobre un espacio de mensajes $\mathcal{M}$ es una tupla $(Setup, Next, Sig, Update, Ver)$ de algoritmos PPT :

- $Setup(\lambda)$: define parámetros del sistema y genera el primer par de claves del sistema (pk_1, sk_1) .
- $Next(pk_e, sk_e)$: recibe el par de claves de una época $e \in \{1, \dots, e_{max}\}$ y devuelve $(sk_{e+1}, pk_{e+1}, \Delta_{e+1})$ donde Δ_{e+1} es el token de actualización de época.
- $Sig(sk_e, M)$: firma el mensaje M con la clave sk_e para la época e y devuelve la firma σ_e .
- $Update(\Delta_{e+1}, M, \sigma_e)$: recibe el token de actualización Δ_{e+1} , el mensaje M y la firma σ_e y, si la época $e < e_{max}$, devuelve (M, σ_{e+1}) .
- $Ver(pk_e, M, \sigma_e)$: devuelve 1 si la firma σ_e es válida para el mensaje M y la clave pública pk_e en la época e . En caso contrario, devuelve 0.

Definición 1.10 (Correctitud de US). Diremos que un esquema de firma US es correcto si $\forall \lambda \in \mathbb{N}$, $M \in \mathcal{M}$, $\forall e, e' \in \{1, \dots, e_{max}\}$ y cumpliéndose:

- $Ver(pk_e, M, \sigma_e) = 1$,
- $(pk_1, sk_1) \leftarrow Setup(\lambda)$,
- $(pk_{e+1}, sk_{e+1}, \Delta_{e+1}) \leftarrow Next(pk_e, sk_e)$,
- $(M, \sigma_{e+1}) \leftarrow Update(\Delta_{e+1}, M, \sigma_e)$

tenemos que $Pr[Ver(pk_{e'}, M, \sigma_{e'}) \neq 1] \leq \epsilon(\lambda)$, donde $\epsilon(\lambda)$ es despreciable en λ . Además, diremos que es perfectamente correcto si $\epsilon(\lambda) = 0$.

Dicho esto, pasamos a explicar el funcionamiento del marco de firma digital que precede a la propuesta presentada en el paper que aquí tratamos.

1.2. CRSST21

Un framework o marco de firma digital, es un conjunto de estándares y protocolos que se agregan a un esquema de firma digital para dotarlo de ciertas propiedades como por ejemplo, agregar condiciones de seguridad o hacer que las firmas sean actualizables. En este caso, describimos el marco **CRSST21** basado en el esquema **GPV**. El problema de fondo de este esquema consiste en encontrar preimágenes cortas de una determinada función lineal modular. Más adelante veremos que se entiende por preimágenes cortas.

En primer lugar, el sistema (cada instancia creada del protocolo) está dividido en épocas. De forma general, para la época e dispondremos de un par de claves (A_e, T_e) y en cada cambio de época, actualizaremos la firma al par (A_{e+1}, T_{e+1}) de manera que si tenemos un mensaje firmado, la misma firma sea válida para las nuevas claves.

Supongamos que nos encontramos en la época e , ilustramos ahora el proceso de actualización de manera esquemática, utilizando el escenario típico de Alice (la firmante) y Bob (el verificador):

- **Setup**(λ): ejecutamos **KeyGen**(1^λ) donde λ es el parámetro de seguridad. Esta función establece los siguientes parámetros:
 - El número máximo de actualizaciones será $N = \text{polylog}(\lambda)$
 - La dimensión del espacio de llegada de nuestra función lineal será $n = \text{poly}(\lambda)$.
 - El entero sobre el que tomamos módulo será $q = n^{\mathcal{O}(N)}$.
 - La dimensión del espacio de partida de nuestra función lineal será $m = \mathcal{O}(n \log q)$.
 - El parámetro gaussiano que utilizaremos será $s = \omega(\sqrt{\log n})$. Más concretamente, se pide que $s \geq \|\bar{B}\| \cdot \omega(\sqrt{\log n})$, donde $\|\bar{B}\|$ es la mayor norma de entre los vectores obtenidos con Gram-Schmidt a partir de T .
 - Por último se fija un parámetro $\beta = \omega(2^N)$ (en cada época) como cota de aceptación de la firma. Sobre esta cota sabemos que debe crecer como mínimo pero su valor concreto se determina en función de los demás parámetros. Más adelante veremos a que nos referimos.

A continuación, introducimos algunos de los parámetros en **TrapGen**(n, m, q, s) para obtener (A_e, T_e) . Estas serán las claves pública y privada de Alice respectivamente.

Sobre la trapdoor impondremos que sea una “base buena” del retículo, es decir, que sean vectores cortos, con buena geometría y Gram-Schmidt controlado (que $\|\bar{B}\|$ sea relativamente pequeño).

- **Sig**(sk_e, m): Alice quiere firmar el mensaje m , por lo que calcula el síndrome $y = H(m|t)$ donde t es un salt que añadimos al mensaje para evitar que el mismo mensaje se firme

de igual manera (se obtiene con un algoritmo típico de generación aleatoria de bits). El problema a resolver que plantea el esquema GPV es encontrar preimágenes cortas del conjunto:

$$\Lambda_{q,u}^\perp(A) = \{u \in \mathbb{Z}^m : Au = y \pmod{q}\}$$

Encontrar preimágenes de esta función es relativamente sencillo. Lo que no es tan sencillo es encontrar preimágenes que sean suficientemente cortas y es ahí donde entra en juego T_e . La trapdoor es una base del conjunto de soluciones homogéneas:

$$\Lambda_q^\perp(A) = \{u \in \mathbb{Z}^m : Au = 0 \pmod{q}\}$$

Notemos que el conjunto de soluciones del problema inicial planteado es una variedad afín de $\Lambda_q^\perp(A)$, por lo que son de la forma $u_0 + \Lambda_q^\perp(A)$ donde u_0 es una solución particular de $Au = y \pmod{q}$. El algoritmo **SamplePre**(A_e, T_e, y, s) lo que hace es calcular una solución particular u_e para el síndrome y y escoge una solución corta del conjunto $u_e + \Lambda_q^\perp(A)$ muestreando según una gaussiana discreta sobre el subespacio afín de soluciones. Se entenderá por solución corta cualquiera cuya norma sea menor que β . Dicho esto, Alice ejecuta la función **SamplePre**(A_e, T_e, y, s) y obtiene la firma u_e (si se usa salt la firma sería (u_e, t)).

- **Next**(A_e, T_e): supongamos que por motivos de seguridad, se requiere que se actualicen las claves de Alice. Para este proceso, se ejecuta la función **Next**(A_e, T_e) para generar un nuevo par de claves (A_{e+1}, T_{e+1}) actualizadas. Lo que se pretende buscar es un Δ_{e+1} (token de actualización) tal que $A_{e+1} \cdot \Delta_{e+1} = A_e \pmod{q}$ y para ello se ejecuta la función **SamplePre**(A_{e+1}, T_{e+1}, s, A_e) ya que, para cada columna de A_e , estaríamos calculando preimágenes cortas muestreadas según la gaussiana discreta (las columnas de Δ_{e+1}).
- **Update**($\Delta_{e+1}, (u_e, t)$): el proceso de actualización de firma se realiza mediante esta función que devuelve $(u_{e+1}, t) = (\Delta_{e+1}u_e, t)$.
- **Ver**(m, σ_e): cuando Bob quiere verificar la firma de Alice, toma la clave pública y la firma más actualizada de Alice, que son A_{e+1} y (u_{e+1}, t) con $u_{e+1} = \Delta_{e+1}u_e$. Para verificar la firma Bob calcula $y = H(m|t)$ y luego comprueba que la firma aportada es, en efecto, una solución corta del problema. Por un lado comprueba que

$$A_{e+1}u_{e+1} = A_{e+1}\Delta_{e+1}u_e = A_eu_e = y \pmod{q}$$

Y por otro lado, verifica que la firma cumple las condiciones de control de norma. La cota $\|u_{e+1}\| \leq \beta$ se cumple porque vamos actualizando β para que soporte un número concreto de actualizaciones.

Lo que parece razonable preguntarse es, ¿de qué manera crece el tamaño de la firma tras cada actualización? Esto es esencial conocerlo ya que determina como ajustar la cota β pues no es algo inmediato. Si desarrollamos la firma actualizada tenemos que

$$\|u_{e+1}\|_2 = \|\Delta_{e+1}u_e\|_2 \leq \|\Delta_{e+1}\|_2 \cdot \|u_e\|_2$$

Notemos que podemos considerar $X = sY$ como una versión escalada por s de una gaussiana Y de escala unitaria. Asimismo, tenemos que $E[X^2] = s^2 E[Y^2] = \mathcal{O}(s^2)$. Por lo tanto, en el caso m -dimensional, tenemos que la norma esperada de nuestro vector aleatorio u_e muestreado según una gaussiana de parámetro s sería:

$$E[\|u_e\|_2^2] = E\left[\sum_{j=1}^m x_j^2\right] = \sum_{j=1}^m E[x_j^2] = \mathcal{O}(ms^2)$$

Dado que la función $f(x) = \sqrt{x}$ es cóncava, podemos aplicar la desigualdad de Jensen para obtener que

$$E[\|u_e\|_2] = E[\sqrt{\|u_e\|_2^2}] \leq \sqrt{E[\|u_e\|_2^2]} = \sqrt{\mathcal{O}(s^2m)} = \mathcal{O}(s\sqrt{m})$$

por lo que nos faltaría acotar $\|\Delta_{e+1}\|_2$. Tal y como se genera el token, sabemos que las columnas de Δ_{e+1} son vectores cortos generados con **SamplePre** al igual que u_e , luego todas las columnas son $\mathcal{O}(s\sqrt{m})$ igualmente. Si usamos que $\|\cdot\|_2 \leq \|\cdot\|_F$ podemos acotar de la siguiente forma:

$$\|\Delta_{e+1}\|_2^2 \leq \|\Delta_{e+1}\|_F^2 = \sum_{j=1}^m \|\delta_j\|_2^2 = m \cdot \mathcal{O}(s\sqrt{m})^2 \implies \|\Delta_{e+1}\|_2 = \sqrt{m} \cdot \mathcal{O}(s\sqrt{m}) = \mathcal{O}(s \cdot m)$$

Y por lo tanto, sabemos que

$$\|u_{e+1}\|_2 = \mathcal{O}(s\sqrt{m})\mathcal{O}(s \cdot m) = \mathcal{O}(s^2 \cdot m^{3/2})$$

En consecuencia, tras r actualizaciones, tendremos que $\|u_{e+r}\|_2 = \mathcal{O}(s^{r+1} \cdot m^{r+\frac{1}{2}})$. Si queremos soportar hasta N actualizaciones, sería conveniente que la cota domine el término anterior, es decir, que $\beta = \omega(s^{N+1} \cdot m^{N+\frac{1}{2}})$. Sea como fuere, conviene recordar que esto es una condición suficiente de correctitud, no una formulación exacta de la cota.

1.3. Análisis de seguridad

Tras presentar el marco de firma **CRSST21**, debemos presentar un análisis de seguridad. De ahora en adelante, cambiaremos la notación $sk_e = T_e$ y $pk_e = A_e$ dado que es el estándar. Antes de nada, en los experimentos de seguridad se manejan unos conceptos que debemos definir formalmente. En primer lugar, cabe aclarar que el objetivo del adversario será conseguir una victoria no trivial en los experimentos. En segundo lugar, presentamos el marco formal[1] en el que se desarrollan estos experimentos.

Definición 1.11. Sea $[e] = \{1, \dots, e\}$, definimos el estado global de un experimento como la tupla $\mathbf{S} = (\mathcal{I}, \mathcal{K}, \mathcal{T}, \mathcal{S})$ donde

$\mathcal{I} = \{((pk_{e'}, sk_{e'}), \Delta_{e'})_{e' \in [e]}\}$: todas las claves y tokens generados.
 $\mathcal{K} = \{e' \in [e]\}$: todas las épocas donde la clave está corrompida.
 $\mathcal{T} = \{e' \in [e]\}$: todas las épocas donde el token está corrompido.
 $\mathcal{S} = \{(e', M, \sigma_{e'})_{e' \in [e]}\}$: todas las tuplas generadas por una firma en la época e' o actualizadas a partir de una firma anterior.

Además, debemos definir una serie de “oráculos” que permitirán al adversario pedir información sobre el sistema, cambiando $\mathbf{S}$ en el proceso.

Definición 1.12. Sea $\mathbf{S}$ el estado global del experimento. Entonces el adversario puede ejecutar las siguientes funciones para recibir información:

- **Sig'**(M, e'): si el mensaje M es válido y estamos en la época $e' \in [e]$, calcula la firma $\sigma_{e'} \leftarrow \text{Sig}(sk_{e'}, M)$, establece $\mathcal{S} := \mathcal{S} \cup \{(e', M, \sigma_{e'})\}$ y devuelve $\sigma_{e'}$. En caso contrario devuelve $\perp$ (valor nulo).
- **Next'**: busca $(sk_e, pk_e) \in \mathcal{I}$, calcula $(pk_{e+1}, sk_{e+1}, \Delta_{e+1}) \leftarrow \text{Next}(pk_e, sk_e)$, establece $\mathcal{I} := \mathcal{I} \cup \{((pk_{e+1}, sk_{e+1}), \Delta_{e+1})\}$, devuelve pk_{e+1} y establece $e := e + 1$.
- **Update'**($M, \sigma_{e'}$): si $\text{Ver}(M, \sigma_{e'}) = 1$, calcula $(M, \sigma_{e'+1}) \leftarrow \text{Update}(\Delta_{e'+1}, M, \sigma_{e'})$, establece $\mathcal{S} := \mathcal{S} \cup \{(e' + 1, M, \sigma_{e'+1})\}$ y devuelve $\sigma_{e'+1}$. En caso contrario, devuelve $\perp$.
- **Corrupt'**($\{token, key\}, e'$): si recibe un token, devuelve $\Delta_{e'+1}$ y establece $\mathcal{T} := \mathcal{T} \cup \{e'\}$ (siempre que $e' < e$). Si recibe una clave, devuelve $sk_{e'}$ y establece $\mathcal{K} := \mathcal{K} \cup \{e'\}$ (siempre que $e' < e$). En caso contrario, devuelve $\perp$.

Nos falta definir lo que se conoce como conjuntos de inferencia. Este concepto sirve para definir nuevos conjuntos a partir de los que construimos en el estado global, recogiendo todas las claves, tokens y firmas que el adversario no obtiene directamente de los oráculos, pero que son fácilmente deducibles a partir de la información disponible. Esto ocurre porque las épocas no son independientes unas de otras sino que están estructuralmente conectadas mediante los tokens de actualización. Por ejemplo, si el adversario pide al oráculo el token de actualización Δ_{e+1} , entonces no solo puede actualizar una firma de la época e , sino que puede actualizar todas las que quiera de la época $e' \in [e]$. Sin embargo, esas firmas que han sido actualizadas no

epoch:	$e-5$	$e-4$	$e-3$	$e-2$	$e-1$	e	$e+1$	$e+2$	$e+3$	$e+4$
keys:	k_{e-5}	k_{e-4}	k_{e-3}	k_{e-2}	k_{e-1}	k_e	k_{e+1}	k_{e+2}	k_{e+3}	k_{e+4}
tokens:	Δ_{e-4}	Δ_{e-3}	Δ_{e-2}	Δ_{e-1}	Δ_e	Δ_{e+1}	Δ_{e+2}	Δ_{e+3}	Δ_{e+4}	Δ_{e+5}
signature:	σ_{e-5}	σ_{e-4}	σ_{e-3}	σ_{e-2}	σ_{e-1}	σ_e	σ_{e+1}	σ_{e+2}	σ_{e+3}	σ_{e+4}

Figura 1.3: Ejemplos de información obtenida directamente (cuadros verdes) e información inferida (cuadros azules). Tabla obtenida de [1].

las podemos incluir en $\mathcal{S}$ porque no han sido explícitamente pedidas, por lo que las añadimos conceptualmente a otro conjunto $\mathcal{S}^*$, al que podemos llamar el cierre de $\mathcal{S}$.

Adicionalmente, se definirán otras condiciones que pueden parecer extrañas a priori. Aunque la actualización hacia el futuro sea necesaria por construcción del esquema, la actualización hacia el pasado no lo es, aunque es potencialmente posible. Supongamos que se conoce sk_e y Δ_e , entonces si el token de actualización resulta ser una matriz invertible, podría calcular sk_{e-1} lo cual sería deducir información en la dirección hacia atrás. Por tanto, si una época está comprometida, también podemos considerar que su época anterior lo está. En cualquier caso esto solo endurece lo que es considerado como victoria trivial¹. Además, reforzar el esquema para impedir que se pueda derivar información hacia atrás no es especialmente útil en la práctica, dado que las firmas antiguas quedan obsoletas y no se pueden usar en un contexto real.

Definición 1.13. Sea US un esquema de firma actualizable con los parámetros habituales, entonces definimos los conjuntos de inferencia de la siguiente forma:

$$\begin{aligned}\mathcal{K}^* &:= \{e \in [N] \mid (e \in \mathcal{K}) \vee (e-1 \in \mathcal{K} \wedge e \in \mathcal{T}) \vee (e+1 \in \mathcal{K} \wedge e+1 \in \mathcal{T})\} \\ \mathcal{T}^* &:= \{e \in [N] \mid (e \in \mathcal{T}) \vee (e-1 \in \mathcal{K}^* \wedge e \in \mathcal{K}^*)\}\end{aligned}$$

¹Con victoria trivial nos referimos a una victoria del adversario en el experimento que se produce gracias a información que ya estaba comprometida.

$$\mathcal{S}^* := \begin{cases} \{(e, M) \mid \text{corrupt-sig}(e, M) = \text{true}\} \text{ donde } \text{corrupt-sig}(e, M) = \text{true} \\ \text{si y solo si: } ((e, M, \cdot) \in \mathcal{S}) \vee (e \in \mathcal{K}^*) \vee (\text{corrupt-sig}(e-1, M) \wedge e \in \mathcal{T}^*) \\ \vee (\text{corrupt-sig}(e+1, M) \wedge e+1 \in \mathcal{T}^*) \end{cases}$$

Con estos conceptos claros, pasamos a definir las dos principales nociones de seguridad. Informalmente, la noción de US-UU-CMA nos indica que un adversario *PPT* no puede distinguir una firma fresca de una actualizada, incluso habiendo visto todas las claves, tokens y firmas de las épocas anteriores. Para poner a prueba este concepto, se define el siguiente experimento:

1. Se inicia el sistema con los parámetros λ y N .
2. Se establece el estado global en su estado inicial $\mathbf{S} = (\{((pk_1, sk_1), \perp)\}, \emptyset, \emptyset, \emptyset)$.
3. El adversario proporciona un par (m^*, e^*) con un mensaje y una etapa objetivo (aquí entra “Chosen Message Attack”).
4. El challenger escoge un bit aleatorio $b \leftarrow \{0, 1\}$.
5. Se generan $\sigma^{(0)} \leftarrow \text{Sig}(sk_{e^*}, m^*)$, $\sigma^{(1)} \leftarrow \text{UpdateCh}(m^*)$, i.e, una firma $\sigma^{(0)}$ fresca de m^* y otra $\sigma^{(1)}$ como resultado de actualizar una firma del mensaje m^* firmado en la etapa $e' \in [e^* - 1]$.
6. El adversario devuelve un bit b^* teniendo pleno acceso a los oráculos.
7. Finalmente, si adversario conoce todas las firmas anteriores de m^* a la etapa objetivo e^* y $b = b^*$ entonces se devuelve 1 (victoria no trivial) y en caso contrario 0 (derrota).

Experiment $\text{Exp}_{\text{US}, \mathcal{A}}^{(\text{w/b})\text{us-uu-cma}}(\lambda, T)$
 $(pk_1, sk_1) \leftarrow \text{Setup}(\lambda, T)$
 $\mathbf{S} = (\mathcal{I}, \mathcal{K}, \mathcal{T}, \mathcal{S})$, for $\mathcal{I} := \{((pk_1, sk_1), \perp)\}, \mathcal{K} := \mathcal{T} := \mathcal{S} := \emptyset$
 $(m^*, e^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sig}}, \mathcal{O}_{\text{Next}}, \boxed{\mathcal{O}_{\text{Upd}}}, \mathcal{O}_{\text{Cor}}}(\lambda, pk_1)$
 $b \leftarrow \{0, 1\}$
 $\sigma^{(0)} \leftarrow \text{UpdateCh}(m^*), \sigma^{(1)} \leftarrow \text{Sig}(sk_{e^*}, m^*)$
 $b^* \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sig}}, \mathcal{O}_{\text{Next}}, \boxed{\mathcal{O}_{\text{Upd}}}, \mathcal{O}_{\text{Cor}}}(\sigma^{(b)})$
 if $(e', m^*, \cdot) \in \mathcal{S}, e' < e^*, e' + 1 = e^*$, and $b = b^*$ then return 1 else return 0

Figura 1.4: Esquema de $\text{Exp}_{\text{US}, \mathcal{A}}^{\text{us-uu-cma}}(\lambda, N)$ extraído de [8].

Evidentemente, se espera que el adversario acierte el 50 % de las veces. No obstante, los requisitos para cumplir esta noción de seguridad son los siguientes.

Definición 1.14 (US-UU-CMA). Diremos que un esquema de firma digital US es US-UU-CMA-seguro si y solo si para cualquier adversario $\mathcal{A}$ que sea PPT^2 si la función de ventaja

$$Adv_{US,\mathcal{A}}^{us-uu-cma}(\lambda, N) := \left| Pr[Exp_{US,\mathcal{A}}^{us-uu-cma}(\lambda, N) = 1] - \frac{1}{2} \right|$$

es despreciable en λ . Es decir, la probabilidad de tener éxito en el experimento $Exp_{US,\mathcal{A}}^{us-uu-cma}$ decae más rápido que la inversa de cualquier polinomio en λ .

Análogamente, podemos definir que la noción de US-EUF-CMA, la cual expresa que el atacante no puede crear una firma que no pudiera obtener de manera trivial de manera directa o inferida de la información aportada por el oráculo. Para poner a prueba este concepto, se define el siguiente experimento:

1. Se inicia el sistema con los parámetros λ y N .
2. Se establece el estado global en su estado inicial $\mathbf{S} = (\{((pk_1, sk_1), \perp)\}, \emptyset, \emptyset, \emptyset)$.
3. El adversario devuelve un par $(m^*, \sigma_{e^*}^*)$ con un mensaje y una firma de ese mensaje para la época e^* (aquí entra “Chosen Message Attack”). Para generar la firma puede consultar toda la información que desee de los oráculos.
4. Finalmente, si el adversario es válido y $Ver(pk_{e^*}, m^*, \sigma_{e^*}) = 1$, entonces se devuelve 1 (victoria no trivial) y en caso contrario se devuelve 0 (derrota).

Se considera que el adversario es válido para este experimento si y solo si $\{e^*, m^*\} \cap \mathcal{S}^* = \emptyset$, es decir, que la firma del mensaje no sea trivial de generar a partir de la información disponible para el adversario.

Experiment $Exp_{US,\mathcal{A}}^{(w/b)us-euf-cma}(\lambda, T)$
 $(pk_1, sk_1) \leftarrow \text{Setup}(\lambda, T)$
 $\mathbf{S} = (\mathcal{I}, \mathcal{K}, \mathcal{T}, \mathcal{S})$, for $\mathcal{I} := \{((pk_1, sk_1), \perp)\}, \mathcal{K} := \mathcal{T} := \mathcal{S} := \emptyset$
 $(m^*, \sigma_{e^*}^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Sig}}, \mathcal{O}_{\text{Next}}, \boxed{\mathcal{O}_{\text{Upd}}}, \mathcal{O}_{\text{Cor}}}(\lambda, pk_1)$
 if $\mathcal{A}$ is valid and $Ver(pk_{e^*}, m^*, \sigma_{e^*}) = 1$ then return 1 else return 0.

Figura 1.5: Esquema de $Exp_{US,\mathcal{A}}^{us-euf-cma}(\lambda, N)$ extraído de [8]

Definición 1.15 (US-EUF-CMA). Diremos que un esquema de firma digital US es US-EUF-CMA-seguro si y solo si para cualquier adversario $\mathcal{A}$ válido que sea PPT si la función de ventaja

$$Adv_{US,\mathcal{A}}^{us-euf-cma}(\lambda, N) := Pr[Exp_{US,\mathcal{A}}^{us-euf-cma}(\lambda, N) = 1]$$

²Probabilistic Polynomial Time: indica que los recursos del adversario crecen polinómicamente con el parámetro de seguridad, lo que hace que el adversario se considere eficiente.

es despreciable en λ . Es decir, la probabilidad de tener éxito en el experimento $Exp_{US,\mathcal{A}}^{us-euf-cma}$ decae más rápido que la inversa de cualquier polinomio en λ .

Ahora cabe preguntarse si el esquema de firma presentado cumple estos requisitos de seguridad. En primer lugar, en [1] se menciona explícitamente que el esquema no es US-UU-CMA-seguro, principalmente porque en el proceso de firma se utiliza un salt t aleatorio, que se recicla en el proceso de actualización, haciendo posible conseguir una ventaja en el experimento simplemente comparando t_e con t_{e-1} . Más aún, de forma general, el problema aparece cuando el proceso de firma es probabilístico pero el proceso de actualización es determinista. Si se plantea el experimento $Exp_{US,\mathcal{A}}^{us-uu-cma}(\lambda, N)$ con este diseño, el adversario solo tendría que actualizar una firma de un mensaje m elegido por el y compararlo con la firma que le aporta el experimento para tomar una decisión. Como el proceso de actualización es determinista, su cálculo debe coincidir con la firma en caso de que fuese actualizada.

En segundo lugar, en [8] también se dice que para $q_{upd,e} \leq n$, CRSST21 es US-EUF-CMA-seguro, donde n es la dimensión del espacio de llegada y $q_{upd,e}$ es el número de actualizaciones para una época e . Sin embargo esta es una cota muy pequeña para un uso real del esquema. En general, uno puede probar que el esquema es US-EUF-CMA-seguro si impide que el adversario ejecute **Update'** (lo que en [8] se conoce como wUS-EUF-CMA). El problema de nuevo es que las actualizaciones son deterministas, por lo que si el adversario se hiciera con m pares (σ_{e-1}, σ_e) linealmente independientes, podría construir las matrices $\mathbf{T} = (\sigma_e^{(1)}, \dots, \sigma_e^{(m)}) \in \mathbb{Z}^{m \times m}$ y $\mathbf{D} = (\sigma_{e-1}^{(1)}, \dots, \sigma_{e-1}^{(m)}) \in \mathbb{Z}^{m \times m}$ para calcular $\Delta_e = \mathbf{T} \cdot \mathbf{D}^{-1}$. De esta manera, el adversario conseguiría forjar una firma de manera no trivial actualizando una firma de un mensaje m^* escogido por él en la etapa e . Sería válido porque dicha firma no se ha pedido directamente al oráculo y además $e-1, e \notin \mathcal{T}^*$.

Antes de terminar el capítulo, es necesario mencionar que, aunque no lo hemos tenido en cuenta en la explicación por simplicidad, en [8] se definen unas versiones debilitadas de estas condiciones de seguridad. Concretamente son (w)US-EUF-CMA y (w)US-UU-CMA donde las versiones con “w” (weaked) eliminan del experimento la posibilidad de usar el oráculo de actualización. Las aportaciones del paper son los siguientes dos resultados.

Teorema 1.16. *Si un esquema de firma US es wUS-UU-CMA-seguro entonces también es US-UU-CMA-seguro. Más aún, para cualquier adversario $\mathcal{A}$ que sea PPT contra la seguridad US-UU-CMA, existe un algoritmo PPT $\mathcal{C}$ tal que*

$$Adv_{US,\mathcal{A}}^{us-uu-cma}(\lambda, N) \leq q_{upd} \cdot (N-1) \cdot Adv_{US,\mathcal{C}}^{wus-uu-cma}(\lambda, N)$$

donde q_{upd} es el número de ejecuciones de actualización de $\mathcal{A}$.

Teorema 1.17. *Si un esquema de firma US es a la vez wUS-UU-CMA-seguro y wUS-EUF-CMA-seguro entonces también es US-EUF-CMA-seguro. Más aún, para cualquier adversario $\mathcal{A}$ que sea PPT contra la seguridad US-UU-CMA, existen unos algoritmos PPT $\mathcal{B}$ y $\mathcal{C}$ tales que*

$$Adv_{US,\mathcal{A}}^{us-euf-cma}(\lambda, N) \leq q_{upd} \cdot Adv_{US,\mathcal{B}}^{wus-uu-cma}(\lambda, N) + Adv_{US,\mathcal{C}}^{wus-euf-cma}(\lambda, N)$$

donde q_{upd} es el número de ejecuciones de actualización de $\mathcal{A}$.

Para la demostración, remitimos a [8], ya que estos resultados escapan al objetivo de este trabajo. En esencia, el análisis de seguridad realizado sigue siendo válido, pero estas versiones facilitan la demostración formal del mismo.

Capítulo 2

Descripción del esquema propuesto

Tras todo lo mencionado, es momento de presentar la nueva propuesta de esquema de firma presentado en [8]. El principal problema que se busca solventar con este nuevo esquema es el filtrado de información en el proceso de actualización, que debe pasar de ser determinista a ser probabilístico.

2.1. Análisis conceptual de las mejoras introducidas

Recordemos que el proceso de actualización en CRSST21 consistía en multiplicar la firma u_e por el token de actualización Δ_{e+1} . Este proceso es determinista, lo cual permite a un atacante replicarlo tantas veces como considere oportuno. Además, si tenemos que la firma u_e sigue aproximadamente una distribución gaussiana esférica, la acción de Δ_{e+1} deforma dicha distribución priorizando algunas direcciones, filtrando así información sobre Δ_{e+1} .

Más formalmente, tenemos la firma u_e muestreada tal que $Cov(u_e) = s_e^2 \cdot \mathcal{I}_m$, siendo $\mathcal{I}_m$ la matriz identidad correspondiente. Ahora, se tiene que $u_{e+1} = \Delta_{e+1} \cdot u_e$ por lo que

$$Cov(u_{e+1}) = Cov(\Delta_{e+1} \cdot u_e) = \Delta_{e+1} Cov(u_e) \Delta_{e+1}^t = s_e^2 \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t$$

Esto quiere decir que la gaussiana esférica va a deformarse según la geometría de $\Delta_{e+1} \cdot \Delta_{e+1}^t$, escalada por un factor s_e . Un adversario con una muestra suficientemente grande puede obtener información sobre Δ_{e+1} tal como direcciones de mayor estiramiento, autovalores, etc. De hecho, el mayor crecimiento posible es $s_e \cdot \sqrt{\lambda_{max}(\Delta_{e+1} \Delta_{e+1}^t)}$ donde $\lambda_{max}(\Delta_{e+1} \Delta_{e+1}^t)$ denota el mayor autovalor de Δ_{e+1} .

En el nuevo esquema utilizan la técnica de convolución presentada en [6]. En esencia, la idea es añadir un ruido gaussiano con la covarianza que falta para hacer el elipsoide (que representa la distribución de Δ_{e+1}) una esfera de nuevo¹. Elegiremos una perturbación gaussiana $r_{e+1} \in \mathbb{Z}^m$ tal que $Cov(r_{e+1}) = s_{e+1}^2 \cdot \mathcal{I}_m - s_e^2 \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t$. Entonces, la actualización de la firma será $u_{e+1} = \Delta_{e+1} \cdot u_e + r_{e+1}$ y como la suma de gaussianas independientes tiene por covarianza la suma directa de cada distribución, tenemos que

¹En casos reales no estamos en tres dimensiones pero es para ejemplificar visualmente lo que ocurre.

$$\text{Cov}(u_{e+1}) = \text{Cov}(\Delta_{e+1} \cdot u_e + r_{e+1}) = \text{Cov}(\Delta_{e+1} \cdot u_e) + \text{Cov}(r_{e+1}) = s_{e+1}^2 \cdot \mathcal{I}_m$$

Y por tanto, recuperamos una gaussiana esférica para la firma actualizada. Como condición necesaria, hay que imponer que $\Sigma = s_{e+1}^2 \cdot \mathcal{I}_m - s_e^2 \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t$ sea semidefinida positiva. Eso significa que $x \Sigma x^t \geq 0$ para todo x no nulo. Si desarrollamos:

$$\begin{aligned} x(s_{e+1}^2 \cdot \mathcal{I}_m - s_e^2 \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t)x^t &= s_{e+1}^2 \cdot x \cdot \mathcal{I}_m \cdot x^t - s_e^2 \cdot x \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t \cdot x^t \\ &= s_{e+1}^2 \cdot \|x\|^2 - s_e^2 \cdot x \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t \cdot x^t \geq 0 \end{aligned}$$

Despejando y dividiendo por $\|x\|^2$:

$$s_{e+1}^2 \geq s_e^2 \cdot \frac{x \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t \cdot x^t}{\|x\|^2} \quad \forall x \neq 0$$

Si se cumple para todos, se cumple en el peor caso:

$$s_{e+1}^2 \geq s_e^2 \max_{x \neq 0} \frac{x \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t \cdot x^t}{\|x\|^2} = s_e^2 \lambda_{\max}(\Delta_{e+1} \cdot \Delta_{e+1}^t)$$

que es equivalente a pedir que

$$s_{e+1} \geq s_e \cdot \|\Delta_{e+1}\|_2$$

Es decir, si pensamos en lo que nos dice esta última condición, es que el parámetro de densidad del ruido gaussiano, tiene que ser lo suficientemente grande como para “envolver” la dirección que más se extiende en la distribución elipsoidal. Esto es muy visual, ya que lo que hacemos en esencia es encerrar la elipse resultante al multiplicar por el token de actualización en una esfera mayor y para eso es necesario que el radio de la nueva esfera sea suficientemente grande. En [8], dicen que el nuevo parámetro gaussiano debe ser $s_{e+1} = (s_e + 1) \cdot \|\Delta_{e+1}\|_2$ como poco. Además, se demuestra que el número de actualizaciones de época máximo está acotado, pero para nuestro caso no es necesario nada más.

Ejemplo 2.1. Vamos a ver un ejemplo, pero sobre $\mathbb{Z}^2$ Supongamos que tenemos la siguiente distribución normal bidimensional

$$X \sim \mathcal{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}\right) = \mathcal{N}(\mu, \Sigma)$$

Ahora, vamos a aplicar la transformación $Y = AX$ donde

$$A = \begin{pmatrix} -1 & 0 \\ -1 & 1 \end{pmatrix}$$

La idea ahora es añadirle a la distribución un ruido para conseguir “esferificar” la distribución Y . Es decir, calculamos $r \sim \mathcal{N}(\mu, s \cdot \Sigma - 3A \cdot A^t)$ y aplicamos la transformación $Y = AX + r$. En la imagen 2.1, podemos ver cada fase de la transformación de la distribución X .

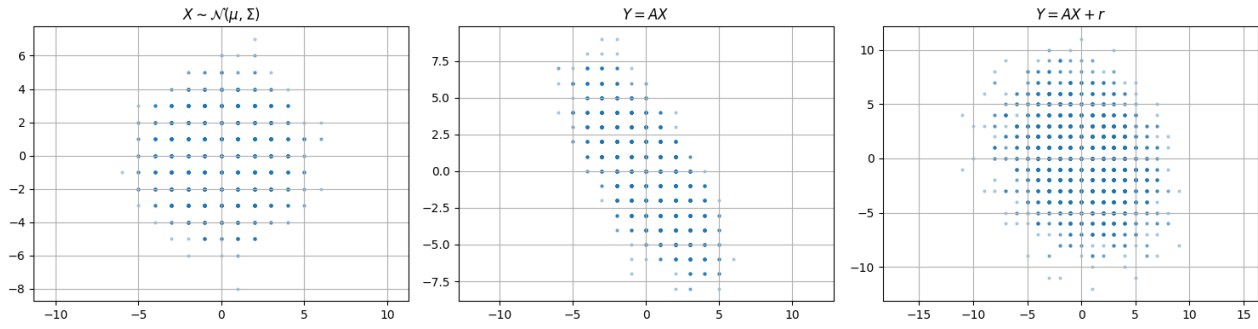


Figura 2.1: Ejemplo de corrección X mediante suma de ruido gaussiano.

Como se puede ver, la distribución se redondea y ya no es tan claro que haya una dirección predominante, ocultando la acción del token de actualización. Esta idea, es la que se formaliza en el siguiente esquema. Hemos tomado el parámetro $s = 2$ para aumentar la dispersión de la actualización lo suficiente, tal y como acabamos de explicar.

2.2. Descripción del nuevo esquema

A continuación definimos el nuevo esquema, en la que usaremos una función “*full-domain-hash*” (FDH) y un esquema auxiliar de cifrado verificable (VE). Un VE es un sistema criptográfico que permite cifrar un testigo w para una afirmación pública x , de modo que cualquiera pueda verificar que el texto cifrado contiene un testigo válido para una relación R , sin revelar el testigo [4]. Necesitamos definir unos conceptos que obtenemos de [8], antes de introducir el nuevo esquema.

Definición 2.2. Sea $q \in \mathbb{Z}$ un entero, $A \in \mathbb{Z}_q^{n \times m}$ una matriz pública y $t \in \mathbb{Z}_q^n$ un vector objetivo. La relación ISIS se define como:

$$R_{ISIS} = \{(A, t, \beta), r : \|r\| \leq \beta, Ar = t \pmod{q}\}.$$

El lenguaje asociado se define como:

$$L_{ISIS} = \{(A, t, \beta) : \exists r \text{ tal que } ((A, t, \beta), r) \in R_{ISIS}\}.$$

Definición 2.3 (Verifiable Encryption). Un esquema de cifrado verificable VE para un lenguaje L es una tupla de algoritmos PPT ($KeyGen, Enc, V, Dec$):

- $KeyGen(1^\lambda)$: define los parámetros necesarios y devuelve un par (vpk, usk) .
- $Enc(vpk, x, w)$: para una afirmación pública x el algoritmo devuelve ct , que resulta de cifrar el testigo w con la clave usk .
- $V(vpk, x, ct)$: el algoritmo devuelve 1 si ct esconde un testigo de x y 0 en caso contrario.
- $Dec(usk, x, ct)$: devuelve el testigo w a partir de ct o devuelve $\perp$.

Dicho esto, procedemos con la presentación del esquema:

- **Setup**(1^λ): esta función establece los siguientes parámetros:
 - El número máximo de actualizaciones será $N = e_{max}$.
 - La dimensión del espacio de llegada de nuestra función lineal será $n = \lambda$.
 - El entero sobre el que tomamos módulo será $q = poly(n)$.
 - El parámetro $r = \omega(\sqrt{\log n})$.
 - La dimensión del espacio de partida de nuestra función lineal será $m = 2n \log q$.
 - El parámetro gaussiano que utilizaremos será $s_1 = 1,6\sqrt{n \log q}$ y luego $s_i := c_{upd} \cdot s_{i-1} + c_{upd} \quad \forall i \in [N] \setminus 1$.
 - La cota de firma será $\beta_1 = 2,3rn \log q$ y luego $\beta_i := c_{upd} \cdot \beta_{i-1} + c_{upd} \cdot \sqrt{m} \quad \forall i \in [N] \setminus 1$.
 - El parámetro $R_i := \{(A, t, \beta_i, r) : \|r\| \leq \beta_i, Ar = t \pmod{q}\} \quad \forall i \in [N]$.
 - Fijamos una función FDH $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q^n$

Finalmente, devolvemos $pp := (\lambda, N, q, n, m, \{s_i\}_{i \in [N]}, \{\beta_i\}_{i \in [N]}, \{R_i\}_{i \in [N]}, H)$ como entrada para los siguientes algoritmos.

- **KeyGen**(pp): generamos $(A_1, T_1) \leftarrow TrapGen(n, m, q)$, $(vsk_1, vpk_1) \leftarrow VE.KeyGen(1^\lambda)$ para el lenguaje L_1 con relación R_1 . Devolvemos $sk_1 = (T_1, vsk_1)$ y $pk_1 = (A_1, vpk_1)$.
- **Sig**(sk_e, m): muestreamos $r_e \leftarrow D_{\mathbb{Z}^m, r \cdot s_e}$, calculamos $t_e = A \cdot r_e \pmod{q}$ y $y = H(m)$. Luego generamos $u_e \leftarrow SamplePre(A_e, T_e, s_e, y + t_e)$. Ciframos el testigo generado $ct_e \leftarrow VE.Enc(vpk_1, (A_e, t_e, \beta_e), r_e)$. Devolvemos $\sigma_e := (u_e, t_e, ct_e)$.
- **Ver**(pk_e, m, σ_e): devuelve 0 si $\|u_e\| > \beta_e$ o si $VE.V(vpk_e, (A_e, t_e, \beta_e), ct_e) = 0$. En caso contrario, calculamos $y = H(m)$ y devolvemos 1 si $A_e \cdot u_e = y + t_e$ o 0 en caso contrario.
- **Next**(pk_e, sk_e): generamos $(A_{e+1}, T_{e+1}) \leftarrow TrapGen(n, m, q)$ y $(vsk_{e+1}, vpk_{e+1}) \leftarrow VE.KeyGen(1^\lambda)$ para el lenguaje L_e con la relación R_e . Muestreamos $\Delta_{e+1} \leftarrow SamplePre(A_{e+1}, T_{e+1}, s_e, A_e)$. Devolvemos $(sk_{e+1} := (T_{e+1}, vsk_{e+1}), pk_{e+1} := (A_{e+1}, vpk_{e+1}), tk_{e+1} := (\Delta_{e+1}, vsk_e, pk_{e+1}))$.
- **Update**(Δ_{e+1}, σ_e): recuperamos $r_e \leftarrow VE.Dec(vsk_e, (A_e, t_e, \beta_e), ct_e)$ y devolvemos $\perp$ si $\|r_e\| > \beta_e$ o $t_e \neq A_e \cdot r_e$. Luego, calculamos $\Sigma = s_{e+1}^2 \cdot \mathbf{I}_m - s_e^2 \cdot \Delta_{e+1} \cdot \Delta_{e+1}^t$, muestreamos $r'_{e+1} \leftarrow D_{\mathbb{Z}^m, r \cdot \sqrt{\Sigma}}$, calculamos $u_{e+1} = \Delta_{e+1} \cdot u_e + r'_{e+1}$, $r_{e+1} = \Delta_{e+1} \cdot r_e + r'_{e+1}$ y $t_{e+1} = A_{e+1} \cdot r_{e+1}$. Ciframos $ct_{e+1} \leftarrow VE.Enc(vpk_{e+1}, (A_{e+1}, t_{e+1}, \beta_{e+1}), r_{e+1})$. Finalmente, devolvemos $sigma_{e+1} = (u_{e+1}, t_{e+1}, ct_{e+1})$.

2.3. Resultados de seguridad

Tras estas modificaciones de CRSST21 en este nuevo esquema al que denotaremos USGPV, los autores de [8], presentan estos dos resultados.

Teorema 2.4. *USGPV es wUS -UU-CMA-seguro.*

Los autores modifican el oráculo de firma, ya que el nuevo esquema solo acepta una única firma por mensaje en cada etapa. Introducen el concepto $uwUS$ -EUF-CMA que, dado que en este esquema la firma es única para cada par (e, m) , resulta equivalente a wUS -EUF-CMA.

Teorema 2.5. *USGPV es $wwUS$ -EUF-CMA-seguro.*

Como consecuencia de los teoremas 1.16 y 1.17, tenemos que USGPV es US -UU-CMA-seguro y US -EUF-CMA-seguro. Para la demostración de estos resultados, remitimos de nuevo a [8], ya que nuestro interés es evaluar el esquema en condiciones reales y no tanto revisar su construcción teórica. No obstante es importante destacar que estas propiedades de seguridad se cumplen en el plano teórico y no para la implementación que desarrollamos a continuación

Capítulo 3

Implementación del esquema

En este capítulo desarrollaremos una prueba de concepto para el esquema USGPV definido en el capítulo anterior. No obstante, deberemos modificar algunos conceptos, ya que muchos de los algoritmos mencionados no son reproducibles tal y como se explican. Resaltamos que en las líneas de código se podrán observar parámetros que poseen definiciones provisionales. Esto se debe a que requieren un estudio especial, que se realizará en el siguiente capítulo.

3.1. Algoritmos adaptados para la implementación

3.1.1. TrapGen

En primer lugar, para implementar **TrapGen** seguiremos la idea presentada en [5]. La idea es elegir una matriz R pequeña que nos permita poder usar una matriz G gadget prefijada con una estructura especial para muestrear preimágenes. De esta forma, el que conozca la matriz R podrá calcular preimágenes de forma sencilla. No obstante, el concepto sigue siendo prácticamente idéntico: tendremos una clave pública A y una privada R que nos permitirá calcular preimágenes cortas.

Definición 3.1. Sea $A \in \mathbb{Z}_q^{n \times m}$ y $G \in \mathbb{Z}_q^{n \times w}$ matrices con $m \geq w \geq n$. Una G -trapdoor para A es una matriz $R \in \mathbb{Z}^{(m-w) \times w}$ tal que $A \begin{bmatrix} R \\ I \end{bmatrix} = HG$ para alguna matriz invertible $H \in \mathbb{Z}_q^{n \times n}$. Llamaremos a H la etiqueta de la trapdoor, cuya calidad se mide por el mayor valor singular $s_1(R)$.

Para nuestro caso, fijaremos $H = I_n$ por simplicidad. Antes de pasar con la implementación del código en sí, es necesario presentar los algoritmos **TrapGen** y **SamplePre** dado que son influidos directamente por el nuevo concepto de trapdoor definido. En primer lugar, el algoritmo **TrapGen** lo describimos de la siguiente forma:

Algorithm 1: TrapGen [5] simplificado para $q = 2^k$

Entrada: Módulo $q = 2^k$, dimensión n y parámetro m .

Salida: Clave pública $A \in \mathbb{Z}_q^{n \times m}$ y trapdoor $R \in \mathbb{Z}^{\bar{m} \times w}$.

1 $k \leftarrow \lceil \log_2 q \rceil$, $w \leftarrow nk$ $\bar{m} \leftarrow m - w$

2 $g \leftarrow (1, 2, 4, \dots, 2^{k-1}) \in \mathbb{Z}_q^{1 \times k}$

3 $G \leftarrow I_n \otimes g \in \mathbb{Z}_q^{n \times w}$

4 Definir $S_k \in \mathbb{Z}^{k \times k}$ como

$$(S_k)_{i,j} = \begin{cases} 2, & i = j, \\ -1, & i = j + 1, \\ 0, & \text{en otro caso.} \end{cases}$$

5 Fijar en el sistema $S \leftarrow I_n \otimes S_k \in \mathbb{Z}^{w \times w}$

6 Muestrear $\bar{A} \leftarrow \mathbb{Z}_q^{n \times \bar{m}}$

7 Muestrear $R \leftarrow \{-1, 0, 1\}^{\bar{m} \times w}$

8 $A_1 \leftarrow G - \bar{A}R$ (mód q)

9 $A \leftarrow [\bar{A} \mid A_1] \in \mathbb{Z}_q^{n \times m}$

10 **return** (A, R)

Con el fin de aportar claridad al esquema, aclaramos que las matrices definidas en los algoritmos anteriores tienen la siguiente forma:

$$G := \begin{pmatrix} \cdots g \cdots & & & \\ & \cdots g \cdots & & \\ & & \ddots & \\ & & & \cdots g \cdots \end{pmatrix} \in \mathbb{Z}^{n \times nk}$$

$$S := \begin{pmatrix} S_k & & & \\ & S_k & & \\ & & \ddots & \\ & & & S_k \end{pmatrix} \in \mathbb{Z}^{nk \times nk}, \quad S_k := \begin{pmatrix} 2 & & & \\ -1 & 2 & & \\ & -1 & \ddots & \\ & & & 2 \\ & & & -1 & 2 \end{pmatrix} \in \mathbb{Z}^{k \times k}$$

3.1.2. SamplePre

En segundo lugar, necesitamos el algoritmo **SamplePre** para muestrear preimágenes del coset de soluciones. Aquí aprovecharemos el nuevo concepto de trapdoor, que nos permitirá hacer uso de unas matrices con una estructura específica para tal fin. Además, introduciremos el parámetro r para escalar el parámetro base de dispersión s_e y cumplir condiciones de correctitud con cierta holgura.

Observemos que por construcción, tenemos que $G \cdot S = 0 \pmod{q}$. Esto nos dice que las columnas de S pertenecen al núcleo modular del retículo $\Lambda^\perp(G)$. Más aún, tenemos el siguiente resultado:

Proposición 3.2. *Sea $q = 2^k$, sea $G = I_n \otimes g$ la matriz gadget, con $g = (1, 2, 4, \dots, 2^{k-1})$, y sea $S = I_n \otimes S_k$, donde S_k es la base gadget definida anteriormente. Entonces*

$$\mathcal{L}(S) = \Lambda_q^\perp(G),$$

es decir, las columnas de S forman una base del retículo homogéneo modular asociado a G .

Demostración. Por construcción, el problema se puede reducir fácilmente a probar que $\mathcal{L}(S_k) = \Lambda_q^\perp(g)$. Entonces, como dijimos antes, por la definición de S_k , sus columnas verifican que $gS_k = 0 \pmod{q}$. Por tanto, esto implica que todo vector de la forma Sc , con $c \in \mathbb{Z}^w$, satisface

$$GSc = 0 \pmod{q}$$

Luego,

$$\mathcal{L}(S) \subseteq \Lambda_q^\perp(G)$$

Nos queda ver que no estamos generando solo una parte del núcleo, sino todo el retículo homogéneo. Consideramos el homomorfismo

$$\varphi : \mathbb{Z}^k \longrightarrow \mathbb{Z}_q \quad \varphi(z) = gz \pmod{q}$$

Como g contiene un 1 en su primera coordenada, esta aplicación es sobreyectiva. Observemos que $\ker(\varphi) = \Lambda_q^\perp(g)$ y por el primer teorema de isomorfía, $\mathbb{Z}^k / \Lambda_q^\perp(g) \cong \mathbb{Z}_q$. Por tanto,

$$[\mathbb{Z}^k : \Lambda_q^\perp(g)] = q$$

Por otro lado, S_k es triangular inferior y todos los elementos de su diagonal son iguales a 2, de modo que

$$|\det(S_k)| = 2^k = q$$

Así,

$$[\mathbb{Z}^k : \mathcal{L}(S_k)] = |\det(S_k)| = q$$

Como ya sabemos que

$$\mathcal{L}(S_k) \subseteq \Lambda_q^\perp(g)$$

y ambos retículos tienen el mismo índice en $\mathbb{Z}^k$, se concluye que

$$\mathcal{L}(S_k) = \Lambda_q^\perp(g)$$

Finalmente, como G y S son matrices diagonales por bloques formadas por g y S_k , respectivamente, el mismo argumento se aplica bloque a bloque. Por tanto,

$$\mathcal{L}(S) = \Lambda_q^\perp(G)$$

En consecuencia, las columnas de S forman una base del retículo homogéneo modular definido por G . □

Este resultado nos permite poder muestrear soluciones $z \leftarrow D_{\Lambda_v^\perp(G), r\sqrt{\Sigma_G}}$ de manera sencilla. El problema a resolver será entonces obtener una preimagen $z \in \mathbb{Z}^w$ tal que $Gz = v$ (mód q). Como $G = I_n \otimes g$, con $g = (1, 2, 4, \dots, 2^{k-1})$, esta ecuación se puede resolver por bloques. En primer lugar, se calcula una solución particular mediante descomposición binaria:

$$z_0 = \text{BitDecomp}(v)$$

donde $\text{BitDecomp}(v)$ devuelve un vector con la concatenación de la codificación de cada una de las coordenadas de v . Esto es sencillo porque G tiene una estructura especial para que sea fácilmente invertible. Sin embargo, usar siempre z_0 produciría una preimagen determinista. Para aleatorizarla sin modificar el síndrome, se utiliza el retículo homogéneo asociado a G :

$$\Lambda_q^\perp(G) = \{y \in \mathbb{Z}^w : Gy = 0 \pmod{q}\}.$$

que como hemos visto, coincide con el retículo generado por S . Por tanto, cualquier vector $y \in \mathcal{L}(S)$ puede añadirse a z_0 sin cambiar el valor de Gz :

$$G(z_0 + y) = Gz_0 + Gy = v \pmod{q}.$$

El objetivo es entonces muestrear un vector homogéneo $y \in \mathcal{L}(S)$ de manera que

$$z = z_0 + y$$

siga el comportamiento de una preimagen gaussiana. Para ello se usa una versión del muestreo de Klein [3]. En primer lugar, tomaremos como centro

$$c = -z_0$$

La razón es que queremos que y esté cerca de $-z_0$, de modo que $z = z_0 + y$ quede cerca del origen y, por tanto, sea corto. El muestreo de Klein procede coordenada a coordenada sobre la base S . Sea

$$S = (s_1, \dots, s_w)$$

y sea

$$(\tilde{s}_1, \dots, \tilde{s}_w)$$

su ortogonalización de Gram-Schmidt. El algoritmo recorre las columnas desde $i = w$ hasta $i = 1$. En cada paso proyecta el centro actual c sobre la dirección ortogonal $\tilde{s}_i$:

$$\mu_i = \frac{\langle c, \tilde{s}_i \rangle}{\|\tilde{s}_i\|^2}$$

Después, dado que $\Sigma_G = \sigma_G I_w$, muestreamos un coeficiente entero

$$a_i \leftarrow D_{\mathbb{Z}, \sigma_i, \mu_i}, \quad \sigma_i = \frac{r\sqrt{\sigma_G}}{\|\tilde{s}_i\|}$$

es decir, una gaussiana discreta unidimensional centrada en μ_i . La elección de σ_i reescala la anchura gaussiana desde el espacio euclídeo al espacio de coordenadas, dado que un paso

unitario en la coordenada i equivale a una distancia $\|\tilde{s}_i\|$, la desviación estándar sobre los coeficientes debe ser $r\sqrt{\sigma_G}/\|\tilde{s}_i\|$. Por eso pedíamos que las columnas de la matriz de Gram-Schmidt tuvieran normas controladas, para que la dispersión dominase la mayor de ellas y σ_i no fuese excesivamente pequeño. A continuación actualiza el centro:

$$c \leftarrow c - a_i s_i$$

Repetimos sucesivamente el proceso y, al finalizar, se obtiene

$$y = \sum_{i=1}^w a_i s_i \in \mathcal{L}(S)$$

y devolvemos $z = z_0 + y$.

Por construcción,

$$Gz = G(z_0 + y) = v \pmod{q},$$

y el uso del centro $-z_0$ hace que la preimagen final tienda a ser corta. Todo este proceso, es el que encerraremos dentro del oráculo $\mathcal{O}(v)$ que aparecerá en el siguiente algoritmo.

Algorithm 2: SamplePre[5] simplificado para $q = 2^k$

Entrada: $\bar{A} \in \mathbb{Z}_q^{n \times \bar{m}}$, $R \in \mathbb{Z}^{\bar{m} \times w}$, oráculo $\mathcal{O}(v)$ para muestrear sobre $\Lambda_v^\perp(G)$ con covarianza $r\sqrt{\Sigma_G} \geq \eta_\epsilon(\Lambda^\perp(G))$, síndrome $u \in \mathbb{Z}_q^n$, $\Sigma_G \geq 2$, $\epsilon \leq 1/2$ y covarianza Σ tal que

$$\Sigma \geq \begin{pmatrix} R \\ I_w \end{pmatrix} (2 + \Sigma_G) \begin{pmatrix} R^t & I_w \end{pmatrix}.$$

Salida: $x \in \mathbb{Z}^m$ a distancia estadística $\mathcal{O}(\epsilon)$ de $D_{\Lambda_v^\perp(A), r\sqrt{\Sigma}}$ tal que $Ax = u \pmod{q}$, para $A = [\bar{A} \mid G - \bar{A}R]$.

- 1 $\Sigma_p \leftarrow \Sigma - \begin{pmatrix} R \\ I_w \end{pmatrix} \Sigma_G \begin{pmatrix} R^t & I_w \end{pmatrix}$
 - 2 $p \leftarrow D_{\mathbb{Z}^m, r\sqrt{\Sigma_p}}$
 - 3 Escribir $p = \begin{pmatrix} p_1 \\ p_2 \end{pmatrix}$, con $p_1 \in \mathbb{Z}^{\bar{m}}$ y $p_2 \in \mathbb{Z}^w$
 - 4 $\bar{w} \leftarrow \bar{A}(p_1 - Rp_2) \pmod{q}$, $w' \leftarrow Gp_2 \pmod{q}$
 - 5 $v \leftarrow u - \bar{w} - w' \pmod{q}$
 - 6 $z \leftarrow \mathcal{O}(v)$ con $z \leftarrow D_{\Lambda_v^\perp(G), r\sqrt{\Sigma_G}}$
 - 7 $x \leftarrow p + \begin{pmatrix} R \\ I_w \end{pmatrix} z$, con $Ax = Ap + Gz = \bar{w} + w' + v = u \pmod{q}$
 - 8 **return** x
-

Aunque hemos explicado el proceso para una entrada vectorial, notemos que al calcular el token de actualización tenemos que usar el mismo algoritmo. Por ello, en la implementación, adaptamos este algoritmo para calcular múltiples preimágenes en paralelo. Simplemente, ajustamos el proceso para que en vez de recibir y devolver un vector, este reciba una matriz

de síndromes y devuelva una matriz de preimágenes. En el caso de un vector, lo tratamos como una matriz de una sola columna.

Hay que aclarar que la garantía de distancia estadística $O(\epsilon)$ del algoritmo anterior se obtiene en el modelo ideal, es decir, cuando el oráculo $\mathcal{O}(v)$ implementa exactamente el muestreo

$$z \leftarrow D_{\Lambda_v^\perp(G), r\sqrt{\Sigma_G}}.$$

En nuestra implementación, este oráculo se aproxima mediante el muestreo de Klein sobre la base S . Sin embargo, tal y como se demuestra en [3], si el parámetro global satisface

$$r\sqrt{\sigma_G} \geq \|\tilde{S}\| \omega(\sqrt{\log w}), \quad \tilde{S} = \max_{1 \leq i \leq w} \|\tilde{s}_i\|$$

tomando $\Sigma_G = \sigma_G I_w$, la distancia estadística entre la distribución producida por dicho sampler y la gaussiana discreta ideal sobre el coset correspondiente es despreciable en w . Por tanto, si P denota la distribución real de nuestra implementación, Q la distribución del algoritmo con oráculo ideal y T la distribución gaussiana ideal final, por la desigualdad triangular se tiene

$$\Delta(P, T) \leq \Delta(P, Q) + \Delta(Q, T) = \text{negl}(w) + O(\epsilon).$$

En consecuencia, bajo las condiciones anteriores, la implementación conserva la cercanía estadística a la distribución ideal. No obstante, el prototipo desarrollado se plantea como una prueba de concepto para entornos controlados, no como una implementación criptográfica desplegable en producción.

3.2. Código del esquema

Para la implementación creamos una clase llamada `usgpv`. Cada instancia de este objeto será un sistema para un usuario concreto donde tendremos definidos los parámetros teóricos (definidos en los algoritmos) y los operativos (necesarios para el desarrollo de la implementación).

Se podrá observar que en muchos métodos de esta clase, no daremos los mismos inputs que los que hemos definido a lo largo de este trabajo, dado que es más natural utilizar los parámetros definidos por el sistema en cada momento. De hecho, esto es necesario porque reproduce la idea de que el usuario solo pueda operar con los parámetros de la época en la que se encuentra.

Antes de pasar con el código, conviene recordar que la definición de algunos parámetros siguen pendientes. Más concretamente, tenemos que definir el parámetro de escala r , el parámetro de dispersión σ_G y la distribución de la matriz R . Para definirlos, tendremos en cuenta las siguientes condiciones:

1. Como explicamos al final de la sección anterior, pedimos que el oráculo realice el muestreo con dispersión $r\sqrt{\Sigma_G} \geq \eta_\epsilon(\Lambda^\perp(G))$. Utilizando el resultado de [3, Lema 3.1] que afirma que

$$\eta_\epsilon(\mathcal{L}(S)) \leq \|\tilde{S}\| \cdot \sqrt{\log(2w(1 + 1/\epsilon))}/\pi,$$

para un ϵ suficientemente pequeño tenemos que

$$r\sqrt{\sigma_G} \geq C \|\tilde{S}\| \sqrt{\log w} \geq \eta_\epsilon(\mathcal{L}(S)) \quad (3.1)$$

Por lo que pidiendo la primera desigualdad, tenemos inmediatamente la segunda. Esta será nuestra primera condición.

2. Por otro lado, pedimos también que Σ_p sea semidefinida positiva, lo que se traduce en que $\Sigma \succeq \begin{pmatrix} R \\ I_w \end{pmatrix} (2 + \Sigma_G) \begin{pmatrix} R^t & I_w \end{pmatrix}$. Si sustituimos con nuestros parámetros, dado que tenemos que $\Sigma = s_e^2 I_m$ y $\Sigma_G = \sigma_G I_w$:

$$s_e^2 I_m \succeq (2 + \sigma_G) B B^t$$

Lo cual se puede reducir a comparar el mayor autovalor de cada lado, obteniendo

$$s_e^2 \geq (2 + \sigma_G) \lambda_{\max}(B B^t)$$

Dado que $B = [R | I_w]^t$, entonces $\lambda_{\max}(B B^t) = \|R\|_2^2 + 1$ y

$$s_e \geq \sqrt{(2 + \sigma_G)(\|R\|_2^2 + 1)} \quad (3.2)$$

Esta será nuestra segunda condición.

3. Y por último, recordemos que en el proceso de actualización, muestreamos un ruido gaussiano de corrección según la matriz

$$\Sigma_{upd} = s_{e+1}^2 I_m - s_e^2 \Delta_{e+1} \Delta_{e+1}^t$$

Teniendo en cuenta que $s_{e+1} = c_{upd} s_e + c_{upd}$:

$$s_{e+1}^2 \geq s_e^2 \|\Delta_{e+1}\|_2 \implies c_{upd} \geq \frac{s_e \|\Delta_{e+1}\|_2}{s_e + 1} \quad (3.3)$$

Esta será nuestra tercera condición.

Antes de pasar Dicho esto, empezamos importando las librerías necesarias, creando la clase `usgpv` y definiendo los parámetros de inicialización. Estos serán los que debe introducir el usuario más algunos parámetros operativos extras.

```

1 import numpy as np
2 import hashlib
3 import math
4 from datetime import datetime
5
6 from VE_scheme import VE_keygen, VE_impostor_encrypt,
  VE_impostor_decrypt

```

```

7 from nacl.public import PublicKey
8
9
10 class usgpv:
11
12     def __init__(self, n_param, N_param, cupd_param, seed=None):
13
14         '''
15         Establecemos los parámetros básicos que debe introducir el
16         usuario, registros y parámetros
17         operativos que utilizaremos en algunas funciones.
18         '''
19
20         # Definimos la época
21         self.epoc_param = 0
22
23         # Definimos la dimensión de salida, el número máximo de
24         # actualizaciones y el parámetro cupd
25         self.n_param = n_param
26         self.N_param = N_param
27         self.cupd_param = cupd_param
28
29         # Definimos las claves privadas, públicas y el token como
30         # variables para usar
31         self.R_matrix = None
32         self.A_matrix = None
33         self.token = None
34         self.sigma_G = 10.0 # Definición provisional
35         self.p0 = 0.04 # Definición provisional
36
37         # Definimos un registro para guardar las claves secretas,
38         # públicas, tokens y firmas por épocas
39         self.secret_keys = []
40         self.public_keys = []
41         self.VE_secret_keys = []
42         self.VE_public_keys = []
43         self.tokens = []
44         self.signs = []
45
46         # Definimos una seed para reproducibilidad y un generador
47         # de números aleatorios de numpy
48         self.seed = seed
49         self.rng = np.random.default_rng(self.seed)

```

Listing 3.1: Creación de la clase usgpv

Seguimos definiendo funciones para registrar claves, tokens y firmas para cada época. También incluimos una función que permite imprimir los parámetros actuales del sistema y una función auxiliar para obtener el Gram-Schmidt de una base cualquiera.

```

1 # Registra una clave secreta
2 def reg_secret_key(self,sk):
3     self.secret_keys.append((self.epoc_param,sk))
4
5 # Registra una clave pública
6 def reg_public_key(self,pk):
7     self.public_keys.append((self.epoc_param,pk))
8
9 # Registra una clave privada de VE
10 def reg_VE_secret_key(self,VE_sk):
11     self.VE_secret_keys.append((self.epoc_param,VE_sk))
12
13 # Registra una clave pública de VE
14 def reg_VE_public_key(self,VE_pk):
15     self.VE_public_keys.append((self.epoc_param,VE_pk))
16
17 # Registra un token de actualización
18 def reg_token(self,token):
19     self.tokens.append((self.epoc_param,token))
20
21 # Registra una firma
22 def reg_sign(self,sign):
23     self.signs.append((self.epoc_param,sign))
24
25 def show_system_params(self):
26
27     '''
28     Esta función imprime los parámetros del sistema para un
29     instante concreto.
30     '''
31     print('\n===== SYSTEM PARAMETERS =====')
32
33     print(f'e          = {self.epoc_param}')
34     print(f'n          = {self.n_param}')
35     print(f'N          = {self.N_param}')
36
37     print(f'q          = {self.q_param}')
38     print(f'k = log2(q) = {self.k_param}')
39
40     print(f'm          = {self.m_param}')
41     print(f'w = nk      = {self.w_param}')
42     print(f'm_bar      = {self.mbar_param}')

```



```

42
43     print(f'r                = {self.r_param}')
44     print(f'c_upd          = {self.cupd_param}')
45     print('='*40)
46     print('A matrix')
47     print(self.A_matrix)
48     print('R matrix')
49     print(self.R_matrix)
50     print('='*40)
51
52     print('\n--- Gaussian parameters s_i ---')
53     print(f's_required_1      = {self.s_required_1}')
54     print(f's_required_2      = {self.s_required_2}')
55     for i, s in enumerate(self.s_params, start=1):
56         print(f's_{i} = {s}')
57
58     print('\n--- Signature bounds beta_i ---')
59     for i, beta in enumerate(self.beta_params, start=1):
60         print(f'beta_{i} = {beta}')
61
62     print('\n=====')
63
64 @staticmethod
65 def gram_schmidt(base: np.ndarray) -> np.ndarray:
66
67     '''
68     Esta función implementa el algoritmo de ortogonalización de
69     Gram-Schmidt
70     Input: base (base de vectores)
71     Output: B_star (base de vectores ortogonales)
72     '''
73     # Tomamos las dimensiones de la base como matriz
74     n, m = base.shape
75
76     # Definimos una matriz de cero en la que iremos guardando las
77     # coordenadas
78     # por columnas
79     B_star = np.zeros((n, m), dtype=float)
80
81     # Iteramos en cada columna de B
82     for i in range(m):
83         # Copiamos la columna en v
84         v = base[:, i].copy()
85
86         # Para cada coordenada de ese vector vertical
87         for j in range(i):

```

```

86
87     # Calculamos el denominador de la proyección
88     denom = np.dot(B_star[:, j], B_star[:, j])
89
90     # Si el denominador es 0 devolvemos error
91     if denom == 0:
92         raise ValueError('La base tiene vectores
93                             linealmente dependientes')
94
95     # Calculamos la proyección de v sobre todos los
96     # vectores que estén ya en B_star
97     projection = (np.dot(base[:, i], B_star[:, j]) / denom)
98     * B_star[:, j]
99
100    # Le restamos a v la parte no ortogonal en cada una de
101    # los vectores de B_star
102    v = v - projection
103
104    # Guardamos v en su columna correspondiente
105    B_star[:, i] = v
106
107    # Devolvemos la base ortogonalizada
108    return B_star

```

Listing 3.2: Funciones auxiliares

A continuación, definimos la función `Setup`, donde calculamos los parámetros derivados de los inputs del usuario

```

1 def setup(self):
2
3     '''
4     Esta función establece el resto de parámetros calculados a
5     partir de los aportados por el usuario.
6     '''
7
8     # Definimos el módulo q
9     self.q_param = int(2**np.ceil(1.3*np.log2(self.n_param)))
10
11    # Definimos la dimensión m del espacio de partida
12    self.m_param = int(2*self.n_param*np.log2(self.q_param))
13
14    # Definimos el parámetro r para escalar dispersión y calcular
15    # cotas.
16    # Definición provisional
17    self.r_param = max(1.0, (np.ceil(np.log2(self.n_param))))

```

```

17 # Calculamos los parámetros gaussianos para todas las épocas
18 # Definición provisional
19 self.s_params =
20     [np.round(1.6*np.sqrt(self.n_param*np.log2(self.q_param)),2)]
21 for _ in range(self.N_param-1):
22     value = self.s_params[-1]
23     self.s_params.append(np.round(self.cupd_param*value +
24                                   self.cupd_param,2))
25
26 # Calculamos las cotas de las firmas para todas las épocas
27 # Definición provisional
28 self.beta_params =
29     [np.round(2.3*self.r_param*self.n_param*np.log2(self.q_param),2)]
30 for _ in range(self.N_param-1):
31     value = self.beta_params[-1]
32     self.beta_params.append(np.round(self.cupd_param*value +
33                                     self.cupd_param*np.sqrt(self.m_param),2))
34
35 # Definimos el parámetro k (número de bits necesarios para
36   enteros módulo q = 2^k)
37 self.k_param = int(np.log2(self.q_param))
38
39 # Definimos el parámetro w
40 self.w_param = int(self.n_param*self.k_param)
41
42 # Definimos el parámetro m barra
43 self.mbar_param = int(self.m_param - self.w_param)

```

Listing 3.3: Definición Setup

Ahora, definimos el algoritmo **TrapGen** según indicamos anteriormente en este mismo capítulo. Recordemos que hay tres condiciones que nuestro esquema debe cumplir, por lo que al definir los siguientes algoritmos, tendremos en mente que estas tres condiciones deben darse simultáneamente. Para ello, pondremos un control en **TrapGen** para las dos primeras condiciones. La tercera, tendrá su control en el algoritmo de actualización de firma.

```

1 def TrapGen(self) -> tuple[np.ndarray, np.ndarray]:
2
3     '''
4     Esta función genera una par (A,R) donde A es una clave pública
5     y R una trapdoor secreta para generar firmas cortas
6     según el esquema usgpv.
7     Input: n_param (dimensión del espacio de salida), m_param
8           (dimensión del espacio de entrada), q_param (módulo)
9     Output: A_e (clave pública), R_e (trapdoor o clave privada)
10    '''

```

```

10 # Definimos el vector g (bloque de la matriz gadget G)
11 self.g_vector = np.array([2**i for i in range(self.k_param)],
12                             dtype=np.int64).reshape(1, self.k_param)
13
14 # Definimos matriz identidad de dimensión n y de dimensión m
15 self.I_n = np.eye(self.n_param, dtype=np.int64)
16 self.I_m = np.eye(self.m_param, dtype=np.int64)
17
18 # Definimos la matriz G gadget como el producto de kron de las
19     dos matrices anteriores
20 self.G_matrix = np.kron(self.I_n, self.g_vector)
21
22 # Ahora definimos la matriz S_k (bloque de la matriz S) como
23     una matriz de ceros
24 self.S_k = np.zeros((self.k_param, self.k_param),
25                     dtype=np.int64)
26
27 # Introducimos los valores correctos en la diagonal y la
28     subdiagonal
29 for i in range(self.k_param):
30     self.S_k[i,i] = 2
31
32     if i < self.k_param-1:
33         self.S_k[i+1,i] = -1
34
35 # Definimos la matriz S como el producto de kron de las matriz
36     S_k y la identidad
37 self.S_matrix = np.kron(self.I_n, self.S_k)
38
39 # Calculamos la matriz de Gram-Schmidt y la guardamos para
40     usarla más adelante
41 self.S_bar = self.gram_schmidt(self.S_matrix)
42
43 # Ahora definimos las matrices A barra aleatoriamente con
44     dimension n x mbar
45 self.A_bar_matrix =
46     self.rng.integers(low=0,high=self.q_param,size=(self.n_param,
47         self.mbar_param),dtype=np.int64)
48
49 # Definimos la matriz R = {-1,0,1}^(mbar x w)
50 # Definición provisional, revisar distribución
51 self.R_matrix = self.rng.choice([-1, 0,
52     1],size=(self.mbar_param, self.w_param),p=[self.p0/2,
53     1-self.p0, self.p0/2]).astype(np.int64)
54
55 # Definimos la segunda parte de la matriz A = G - Abar @ R

```

```

44     self.A1 = (self.G_matrix - self.A_bar_matrix @ self.R_matrix) %
        self.q_param
45
46     # Construimos la matriz A = [Abar | A_1]
47     self.A_matrix = np.concatenate([self.A_bar_matrix, self.A1],
        axis=1)
48
49     # Definimos la identidad de dimensión w x w para construir la
        matriz B = [R | I_w]^t
50     self.I_w = np.eye(self.w_param, dtype=np.int64)
51     self.B_matrix = np.vstack([self.R_matrix,
        np.eye(self.w_param)]).astype(np.int64)
52
53
54     # Definimos las matrices de covarianza Sigma_G y Sigma, ya que
        se usan en varios algoritmos y una vez
55     # fijados los parámetros gaussianos y el parámetro r, podemos
        calcularlas. Además, sigma_G es fijo.
56     self.SIGMA_G = self.sigma_G * self.I_w.astype(float)
57
58     self.SIGMA = (
59         self.s_params[self.epoc_param]
60     )**2 * np.eye(self.m_param, dtype=float)
61
62
63     # Ahora vamos a comprobar si se cumplen las condiciones sobre
        el parámetro de dispersión con las matrices que acabamos
64     # de definir. Se tienen que cumplir dos condiciones:
65
66     # La primera es que s_e >= C*||Sbar||*sqrt(log w) con C > 1.
        Como tomamos provisionalmente sigma_G = 10 y C = 1.5,
67     # realizamos así el cálculo
68     Sbar_max = np.max(np.linalg.norm(self.S_bar, axis=0))
69     self.s_required_1 = 1.5 * Sbar_max *
        np.sqrt(np.log2(self.w_param))
70
71     # Comprobamos si se cumple la primera condición
72     test1 = self.r_param * np.sqrt(self.sigma_G) >=
        self.s_required_1
73     if not test1:
74         raise ValueError('El parámetro de dispersión no supera
            s_required_1')
75
76     print(f'Test de dispersión adecuada para la matriz G: {test1}')
77
78     # La segunda es que s_e >= sqrt((2 + sigma_G)*lambda_max) donde

```

```

79     lambda_max es el mayor valor autovalor de B @ B^t
    lambda_max = np.linalg.eigvalsh(self.B_matrix.T @
    self.B_matrix).max()
80     self.s_required_2 = np.sqrt((2.0+self.sigma_G)*lambda_max)
81
82     # Comprobamos si se cumple la segunda condición
83     test2 = self.s_params[self.epoc_param] >= self.s_required_2
84     if not test2:
85         raise ValueError('El parámetro de dispersión no supera
            s_required_2')
86     print(f'Test de dispersión adecuada para s_{self.epoc_param}:
            {test2}')
87
88     return self.A_matrix, self.R_matrix

```

Listing 3.4: Definición de TrapGen

Pasamos con la definición de `SamplePre`, para la cual necesitaremos definir previamente `BitDecomp`, `klein_sampler` y `oracle_sampler`.

```

1 def BitDecomp(self, V: np.ndarray) -> np.ndarray:
2
3     '''
4     Calcula la descomposición binaria de cada vector columna de la
        matriz de entrada V y
5     la devuelve en una matriz Z_0, donde la columna i de Z_0 es la
        descomposición binaria
6     de la columna i de V.
7
8     Input: V (matriz entera de dimensión n x m)
9     Output: Z_0 (matriz binaria de dimensión nk x m)
10    '''
11
12    # Imponemos que la entrada sea una matriz entera módulo q
13    V = np.asarray(V, dtype=np.int64) % self.q_param
14
15    # Si la entrada es un vector, la convertimos en una matriz
        columna
16    if V.ndim == 1:
17        V = V.reshape(-1, 1) # -1 calcula el número de filas
            automáticamente. El 1 indica la columna
18
19    # Verificamos que la dimensión de salida coincide con la
        establecida en el sistema
20    if V.shape[0] != self.n_param:
21        raise ValueError(f'V debe tener {self.n_param} filas, pero
            tiene shape {V.shape}')

```

```

22
23     # Generamos un vector con tantas coordenadas como bits
24     necesitamos (necesitamos k bits)
25     shifts = np.arange(self.k_param, dtype=np.int64)
26
27     # Coordenada a coordenada combinamos los elementos de cada
28     matriz desplazando i in {1,...,k} bits y
29     # tomando el primero con & 1. Concretamente, se genera bits con
30     # dimensión (V.shape[0],shifts.shape[0],V.shape[1])
31     # y luego se rellena coordenada a coordenada haciendo la
32     # operación que en este caso es
33     #
34     # bits[i,k,j] = (V[i,j] >> shifts[k]) & 1
35
36     bits = (V[:, None, :] >> shifts[None, :, None]) & 1
37
38     # Luego cambiamos a la dimensión que queremos que es nk x m, y
39     # rellenamos por filas, lo cual nos devuelve los bits
40     # de las coordenadas de cada columna concatenados desde el bit
41     # menos significativo de la primera coordenada hasta
42     # el más significativo de la última coordenada
43     Z0 = bits.reshape(self.w_param, V.shape[1]).astype(np.int64)
44
45     return Z0
46
47 def klein_sampler(self, Z0: np.ndarray, S: np.ndarray, sigma=1.0)
48     -> np.ndarray:
49
50     '''
51     Muestrea preimágenes del coset de soluciones  $Gz = 0 \pmod{q}$ .
52     Input: Z0 (preimagenes deterministas), S (base del retículo de
53     soluciones de  $Gz = 0$ ) y sigma (desviación típica de la
54     distribución)
55     Output: Y (soluciones muestreadas según una normal de parámetro
56     sqrt(sigma) de  $Gz = 0$ )
57     '''
58
59     # Empezamos convirtiendo la matriz S a una matriz entera por
60     # seguridad
61     S = np.asarray(S, dtype=np.int64)
62
63     # Guardamos una versión con reales para poder calcular su
64     # Gram-Schmidt
65     S_float = S.astype(float)

```

```

56     # Tomamos las preimagenes deterministas calculada con
        binary_coordinates_matrix
57     Z0 = np.asarray(Z0, dtype=float)
58
59     # Si lo que recibimos es un vector, lo pasamos a una matriz de
        una columna
60     if Z0.ndim == 1:
61         Z0 = Z0.reshape(-1, 1) # Aquí -1 calcula el número de filas
            necesarias automáticamente
62
63     # Comprobamos si ese número de filas es el adecuado según el
        esquema
64     if Z0.shape[0] != self.w_param:
65         raise ValueError(
66             f'Z0 debe tener {self.w_param} filas, pero tiene shape
                {Z0.shape}'
67         )
68
69     # Guardamos el número de columnas, es decir, el número de
        preimágenes que vamos a muestrear
70     L = Z0.shape[1]
71
72     # Calculamos el Gram-Schmidt de la matriz S
73     Sbar = self.gram_schmidt(S_float)
74
75     # Definimos nuestros centros
76     C = -Z0.copy()
77
78     # Definimos la matriz de coeficientes que vamos a ir rellenando
79     A_coeff = np.zeros((self.w_param, L), dtype=np.int64)
80
81     # Vamos a ir muestreando en paralelo las coordenadas de todas
        las preimágenes que necesitamos
82     # desde i = w hasta i = 1
83     for i in reversed(range(self.w_param)):
84
85         # Tomamos la primera columna de la matriz S y Sbar
86         sbar_i = Sbar[:, i]
87         s_i = S_float[:, i]
88
89         # Calculamos el denominador de la media para la fila i
90         norm_sq = np.dot(sbar_i, sbar_i)
91
92         # Comprobamos que la norma no sea nula
93         if norm_sq == 0:
94             raise ValueError('Vector Gram-Schmidt nulo')

```



```

95
96     # Calculamos el vector medias para la fila i
97     mu_i = (sbar_i @ C) / norm_sq
98
99     # Calculamos la varianza para la fila i
100    sigma_i = self.r_param * sigma / np.sqrt(norm_sq)
101
102    # Muestreamos L enteros a la vez con los parámetros que
103    # hemos calculado
104    a_i = np.rint(
105        self.rng.normal(
106            loc=mu_i,
107            scale=sigma_i,
108            size=L
109        )
110    ).astype(np.int64) # Además los redondeamos ya que deben
111    # ser coeficientes enteros
112
113    # Introducimos los coeficientes de la fila i en la matriz
114    # de coeficientes
115    A_coeff[i, :] = a_i
116
117    # Actualizamos todos los centros a la vez. Outer crea un
118    # vector con los productos
119    # de las coordenadas de s_i y a_i
120    C = C - np.outer(s_i, a_i)
121
122    # Una vez calculados todos los coeficientes, multiplicamos por
123    # S y obtenemos todas las
124    # soluciones del coset de soluciones de Gz = 0
125    Y = S @ A_coeff
126
127    return Y.astype(np.int64)
128
129 def oracle_sampler(self, V: np.ndarray) -> np.ndarray:
130
131     '''
132     Esta función calcula preimágenes de las columnas de la matriz V
133     para la ecuación modular
134
135         
$$GZ = V \pmod{q}$$

136
137     muestreando sobre el coset de soluciones con una distancia
138     estadística del muestreo ideal
139     despreciable en el número de columnas de G.
140     Input: V (matriz de síndromes)
141     Output: Z (matriz de preimágenes de la forma  $Z_0 + Y$ )
142     '''

```

```

134
135     # Convertimos la entrada en una matriz entera por seguridad
136     V = np.asarray(V, dtype=np.int64) % self.q_param
137
138     # Si la entrada es un vector, convertimos en una matriz de una
139     #     columna
140     if V.ndim == 1:
141         V = V.reshape(-1, 1)
142
143     # Calculamos la parte determinista
144     Z0 = self.BitDecomp(V)
145
146     # Calculamos la parte probabilística
147     Y = self.klein_sampler(
148         Z0,
149         self.S_matrix,
150         sigma=np.sqrt(self.sigma_G)
151     )
152
153     # Combinamos las soluciones en una matriz de preimágenes para
154     #     la ecuación  $GZ = V \pmod{q}$ 
155     Z = Z0 + Y
156
157     # Comprobamos si efectivamente verifica la ecuación
158     if np.any((self.G_matrix @ Z) % self.q_param != V %
159               self.q_param):
160         raise ValueError('Las preimágenes no cumplen la ecuación  $GZ$ 
161                            $= V \pmod{q}$ ')
162
163     return Z.astype(np.int64)
164
165 def SamplePre(self, Y: np.ndarray, SIGMA_override=False) ->
166     np.ndarray:
167
168     '''
169     Esta función toma una matriz de síndromes y calcula preimágenes
170     para la ecuación modular  $AU = y$ .
171     Input: Y (matriz de síndromes)
172     Output: U (matriz de preimágenes o firmas)
173     '''
174
175     # Convertimos la matriz de entrada en una matriz de enteros por
176     #     seguridad
177     Y = np.asarray(Y, dtype=np.int64) % self.q_param
178
179     # Con esta variable detectamos si la entrada tiene una o más

```

```

173     columnas, es decir, si vamos a calcular
174     # una o más preimágenes. En general, usaremos una preimagen
175     para firmar mensajes y varias preimágenes
176     # para calcular el token de actualización
177     single_input = False
178
179     # Si la entrada es un vector, lo convertimos a una matriz de
180     una columna
181     if Y.ndim == 1:
182         Y = Y.reshape(-1, 1)
183         single_input = True # Modificamos la variable para usarla
184         después
185
186     # Nos aseguramos que la matriz de entrada tenga la dimensión
187     del espacio de salida
188     if Y.shape[0] != self.n_param:
189         raise ValueError(
190             f'Y debe tener {self.n_param} filas, pero tiene shape
191             {Y.shape}'
192         )
193
194     # Convertimos la matriz B a real para usarla en el cálculo de
195     la dispersión del vector p
196     B_float = self.B_matrix.astype(float)
197
198     SIGMA = self.SIGMA
199     # Aquí distinguimos según si queremos generar el token de
200     actualización o una matriz de preimágenes
201     if SIGMA_override:
202         SIGMA = (self.s_params[0])**2 * np.eye(self.m_param,
203             dtype=float)
204
205     # Calculamos la matriz de covarianza para muestrear la
206     propuesta de firma inicial. En este caso, sabemos
207     # que es semidefinida positiva por la segunda condición
208     evaluada en TrapGen
209     self.SIGMA_p = SIGMA - B_float @ self.SIGMA_G @ B_float.T
210     self.SIGMA_p = (self.SIGMA_p + self.SIGMA_p.T) / 2
211
212     # Sampleamos el vector p de una distribución normal
213     multivariante centrada en 0 y con la covarianza SIGMA_p
214     p_real = self.rng.multivariate_normal(
215         mean=np.zeros(self.m_param),
216         cov=(self.r_param**2) * self.SIGMA_p,
217         size=Y.shape[1]
218     ).T

```

```

207
208
209 # Redondeamos p a un vector entero
210 P = np rint(p_real).astype(np.int64)
211
212 # Dividimos el vector p en p1 y p2
213 P1 = P[:self.mbar_param, :]
214 P2 = P[self.mbar_param:, :]
215
216 # Calculamos wbar y wprima
217 Wbar = (
218     self.A_bar_matrix @ (P1 - self.R_matrix @ P2)
219 ) % self.q_param
220
221 Wprima = (
222     self.G_matrix @ P2
223 ) % self.q_param
224
225 # Calculamos V como la diferencia entre Y y la imagen del
226     vector P. Así al calcular la preimagen de V, podremos
227 # corregir P para que verifique la ecuación modular
228 V = (Y - Wbar - Wprima) % self.q_param
229
230 # Sampleamos la preimagen correctora con el oráculo
231 Z = self.oracle_sampler(V)
232
233 # Calculamos finalmente nuestra preimagen
234 X = P + self.B_matrix @ Z
235
236 # Comprobamos que la solución verifica efectivamente la
237     ecuación modular
238 if not np.array_equal((self.A_matrix @ X) % self.q_param, Y):
239     raise ValueError('Las preimágenes generadas no cumplen la
240         ecuación matricial')
241
242 # Si la entrada es un vector, devolvemos la primera columna en
243     forma de vector
244 if single_input:
245     return X[:, 0]
246
247 return X

```

Listing 3.5: Construcción de SamplePre

Antes de pasar a implementar las funciones del esquema en si, nos falta definir una función hash que convierta un mensaje en texto plano a un vector de la dimensión adecuada. La idea

de esta función es generar tantos bits como necesitemos a partir del mensaje de entrada y luego agruparlos en paquetes de longitud k para representarlos como enteros módulo q .

```

1 def H(self, message: str) -> np.ndarray:
2
3     '''
4     Esta función traduce un mensaje en texto plano a un vector de
5     dimensión n con coordenadas módulo q
6     Input: m (mensaje en texto plano)
7     Output: y (vector n dimensional de coordenadas módulo q)
8     '''
9
10    # Verificamos que la entrada es una cadena de texto
11    if isinstance(message, str):
12        # La codificamos con utf-8 para tratar acentos y caracteres
13        especiales
14        message = message.encode("utf-8")
15
16    # Calculamos el número total de bytes que sería w/8 = n*k/8
17    total_bytes = math.ceil(self.w_param / 8)
18
19    # Shake256 es una función hash que, a diferencia de sha_256,
20    # devuelve tantos bits como queramos. Estos
21    # bits están generados a partir del mensaje de entrada.
22    digest = hashlib.shake_256(message).digest(total_bytes) # Nos
23    devuelve los bytes que necesitamos
24
25    # Convertimos la cadena de bytes a un entero para después tomar
26    # la cadena de bits del tamaño que queramos
27    X = int.from_bytes(digest, byteorder="little")
28
29    # Mask va a ser siempre una cadena de k 1's
30    mask = self.q_param - 1
31
32    # Definimos una lista para guardar las coordenadas del síndrome
33    y = H(m)
34    coords = []
35
36    # Calculamos cada coordenada tomando cadenas de k bits y
37    # guardandolas como enteros
38    for i in range(self.n_param):
39        # Para cada coordenada i = 1,...,n , nos desplazamos al bit
40        # en la posición n*k y
41        # tomamos los k bits siguientes usando la máscara definida
42        coord = (X >> (i * self.k_param)) & mask
43        # Después añadimos la coordenada a la lista. Resaltar que

```

```

36     para python a un entero
37     # se le pueden hacer operaciones bit a bit pero seguirá
38     saliendo como un entero
39     coords.append(coord)

return np.array(coords, dtype=np.int64)

```

Listing 3.6: Construcción de la función hash H

Resaltamos que hemos definido un VE diferente. En nuestro caso, `VE_impostor` va a validar el testigo descifrándolo y comprobando internamente que verifica la relación definida. La idea es que un VE realiza este proceso sin la necesidad de descifrar el testigo. Esta decisión la tomamos ya que, aunque existen esquemas diseñados para tal efecto, desarrollar dichos esquemas para la aportación que supone, excede el alcance de este trabajo.

```

1 def VE_impostor_KeyGen(self) -> PublicKey:
2
3     '''
4     Esta función genera unas claves para el VE_impostor
5     Input: -
6     Output: VE_pk (clave pública VE), VE_sk (clave privada VE)
7     '''
8     self.VE_pk, self.VE_sk = VE_keygen()
9
10    self.reg_VE_secret_key(self.VE_sk)
11    self.reg_VE_public_key(self.VE_pk)
12
13    # Devolvemos solo la clave pública, la idea es que solo alguien
14    # con acceso al sistema
15    # tenga la clave privada (por representar el concepto de manera
16    # aproximada sin usar un esquema ZKP)
17    return self.VE_pk
18
19 def VE_impostor_ver(self,vepk: PublicKey,pk: np.ndarray,sign:
20 tuple[np.ndarray,np.ndarray,bytes]) -> bool:
21
22     '''
23     Esta función es una sustituta de la función de verificación de
24     la afirmación pública para
25     el esquema VE. Sirve como recreación de lo que pasaría en una
26     implementación real, pero en
27     un entorno controlado.
28     Input: vepk (clave pública que usaría un usuario para verificar
29     si se cumple la relación),
30     pk (clave pública), sign (firma que contiene el testigo
31     cifrado y la afirmación pública)
32     Output: True/False

```

```

26     '''
27
28     # Usamos la clave pública aportada por el usuario y vemos si
29     # coincide con la que tiene el sistema
30     # asociado a la clave privada de esta época. Comparamos vs la
31     # clave pública registrada para la
32     # época actual. La clave privada de esa misma época es la que
33     # usamos para descifrar
34     if vepk != self.VE_public_keys[self.epoc_param][1]:
35         return False
36
37     # El sistema descifra internamente el testigo cifrado y
38     # comprueba si se cumple la relación
39     witness_e = VE_impostor_decrypt(sign[2],
40                                     self.VE_secret_keys[self.epoc_param][1])
41     if not np.array_equal(sign[1] % self.q_param, (pk @ witness_e) %
42                                     self.q_param):
43         return False
44
45     # Comprobamos también que el testigo cumple la condición de la
46     # cota
47     if np.linalg.norm(witness_e) >
48         self.beta_params[self.epoc_param]:
49         return False
50
51     # Si todo se cumple devolvemos True. En caso contrario,
52     # devolvemos False
53     return True

```

Listing 3.7: Definición de VE_impostor_KeyGen y VE_impostor_ver

Por último, definimos las funciones de generación de claves, firma, verificación, cambio de época y actualización de firma.

```

1 def KeyGen(self) -> tuple[np.ndarray, np.ndarray]:
2
3     '''
4     Esta función genera un par de claves pública/privada haciendo
5     uso del algoritmo TrapGen
6     Input: -
7     Output: pk (clave pública), sk (clave privada)
8     '''
9
10    # Generamos un par de claves y las registramos
11    self.pk, self.sk = self.TrapGen()
12    self.reg_secret_key(self.sk)
13    self.reg_public_key(self.pk)

```

```

13     return self.pk, self.sk
14
15 def Sig(self, m):
16     '''
17     Esta función firma un mensaje o una lista de mensajes haciendo
18     uso del algortimo SamplePre
19     Input: m (mensaje en texto plano o lista de mensajes)
20     Output: sign (firma del mensaje)
21     '''
22     # Detectamos si la entrada es una lista de mensajes o un único
23     # mensaje
24     es_lista = isinstance(m, (list, tuple))
25
26     if es_lista:
27         mensajes = list(m)
28     else:
29         mensajes = [m]
30
31     L = len(mensajes)
32
33     # Si la lista está vacía, devolvemos una lista vacía
34     if L == 0:
35         return []
36
37     # Calculamos todos los síndromes H(m_i)
38     # Y tendrá dimensión n x L
39     Y = np.column_stack([self.H(mensaje) for mensaje in mensajes])
40
41     # Sampleamos todos los testigos a la vez
42     # witness_real tendrá dimensión m_param x L
43     witness_real = self.rng.multivariate_normal(
44         mean=np.zeros(self.m_param),
45         cov=(self.r_param**2) * self.SIGMA,
46         size=L
47     ).T
48
49     # Redondeamos los testigos
50     witness_e = np rint(witness_real).astype(np.int64)
51
52     # Calculamos todas las afirmaciones públicas
53     # statement_e tendrá dimensión n x L
54     statement_e = (self.pk @ witness_e) % self.q_param
55
56     # Calculamos todas las firmas como preimágenes
57     # U tendrá dimensión m_param x L
58     U = self.SamplePre((Y + statement_e) % self.q_param)

```



```

57
58 # Calculamos ahora la lista de firmas
59 signs = []
60
61 # vamos añadiendo cada firma, con su afirmación pública y su
62   testigo cifrado
63 for j in range(L):
64     sign = (
65         U[:, j],
66         statement_e[:, j],
67         VE_impostor_encrypt(witness_e[:, j], self.VE_pk)
68     )
69
70     self.reg_sign(sign)
71     signs.append(sign)
72
73 # Si la entrada original era un único mensaje, devolvemos una ú
74   nica firma
75 if not es_lista:
76     return signs[0]
77
78 # Si era una lista, devolvemos la lista completa
79 return signs
80
81 def Ver(self,vepk: PublicKey,pk: np.ndarray,m: str,sign:
82   tuple[np.ndarray,np.ndarray,bytes]) -> bool:
83
84     '''
85     Esta función verifica que una firma es válida en la época actual
86     Input: vepk (calve pública esquema VE), pk (clave pública), m
87         (mensaje en texto plano),
88         sign (firma del mensaje m)
89     Output: True/False
90     '''
91
92     # Calculamos la norma de la firma
93     norma = np.linalg.norm(sign[0])
94
95     # Comprobamos que la firma cumple la cota
96     if limit := (norma <= self.beta_params[self.epoc_param]):
97         print(f'El vector tiene norma {norma} que es menor que la
98             cota {self.beta_params[self.epoc_param]}')
99
100     # Comprobamos que la afirmación pública es cierta
101     if VE_condition := self.VE_impostor_ver(vepk,pk,sign):
102         print(f'La afirmación pública t_e es cierta para la

```

```

    relación definida')
98
99 # Calculamos el síndrome del mensaje
100 y = (self.H(m) + sign[1]) % self.q_param
101
102 # Comprobamos si el síndrome calculado del mensaje m y la firma
    aportada verifican la ecuación modular
103 if congruence := np.all((pk @ sign[0]) % self.q_param == y):
104     print(f'La congruencia es verdadera')
105
106 # Si todo es cierto devolvemos True. En caso contrario,
    devolvemos False
107 return limit and congruence and VE_condition
108
109
110 def Next(self, pk: np.ndarray) -> tuple[np.ndarray, np.ndarray,
    np.ndarray]:
111
112     '''
113     Esta función pasa el sistema a la siguiente época. Para ello,
        genera unas nuevas claves, ajusta los parámetros
114     de dispersión y actualiza el contador de época.
115     Input: pk (clave pública época e)
116     Output: token (token de actualización para pasar de e -> e+1)
117     '''
118
119     # Nos aseguramos que se ha introducido la clave pública de la é
        poca actual
120     if not np.array_equal(pk % self.q_param, self.pk %
        self.q_param):
121         raise ValueError('La pk introducida no coincide con la
            clave pública actual')
122
123     # Nos aseguramos de que se puede actualizar. Sumamos 1 porque
        permitimos N épocas
124     # desde el 0
125     if self.epoc_param + 1 >= self.N_param:
126         raise ValueError('No puedes actualizar más')
127
128     # Aumentamos la época
129     self.epoc_param += 1
130
131     # Generamos unas nuevas claves y las guardamos en las variables
        del sistema.
132     # Recordemos que internamente se registran las claves con la é
        poca, por eso

```

```

133     # hay que actualizar la época antes.
134     self.pk, self.sk = self.KeyGen()
135
136     # Generamos unas nuevas claves para el esquema VE. La clave
137     # secreta se actualiza
138     # internamente en VE_impostor_KeyGen()
139     self.VE_pk = self.VE_impostor_KeyGen()
140
141     # Calculamos el token de actualización usando el algoritmo
142     # SamplePre
143     token = self.SamplePre(pk, SIGMA_override=True)
144
145     # Si el token verifica la ecuación modular, cambiamos de época
146     if np.array_equal((self.pk @ token) % self.q_param, pk %
147                       self.q_param):
148
149         # Registramos el token
150         self.reg_token(token)
151
152         # Calculamos la nueva matriz de covarianzas para el
153         # parámetro gaussiano
154         # de la siguiente época (s_{e+1})
155         self.SIGMA = (
156             self.s_params[self.epoc_param]
157             )**2 * np.eye(self.m_param, dtype=float)
158
159         # # Mostramos los parámetros del sistema actualizados
160         # self.show_system_params()
161
162         # Devolvemos las claves y el token
163         return self.pk, self.sk, token
164     else:
165         raise ValueError('El token no verifica la condición
166                           algebraica')
167
168 def Update(self, token: np.ndarray, sign: tuple[np.ndarray,
169         np.ndarray, bytes]) -> tuple[np.ndarray, np.ndarray, bytes]:
170
171     # Nos aseguramos de que exista una época anterior
172     if self.epoc_param == 0:
173         raise ValueError('No podemos actualizar en la época
174                           inicial')

```

```

172 # Recuperamos la afirmación pública con la clave privada del
    esquema VE de la época anterior
173 witness_e_old = VE_impostor_decrypt(sign[2],
    self.VE_secret_keys[self.epoc_param - 1][1])
174
175 # Verificamos si cumple la cota definida en la relación
176 norm_witness_e = np.linalg.norm(witness_e_old)
177 if norm_witness_e > self.beta_params[self.epoc_param - 1]:
178     raise ValueError('El testigo no cumple la cota de la norma')
179
180 # Tomamos la clave pública de la época anterior
181 _, last_pk = self.public_keys[self.epoc_param - 1]
182
183 # Verificamos que el token introducido es el token de
    transición entre la etapa anterior y la actual
184 if not np.array_equal((self.pk @ token) % self.q_param, last_pk
    % self.q_param):
185     raise ValueError('El token no actualiza correctamente de
    A_e a A_{e+1}')
186
187 # Verificamos si el testigo cumple la ecuación definida en la
    relación
188 if not np.array_equal((last_pk @ witness_e_old) %
    self.q_param, sign[1] % self.q_param):
189     raise ValueError('El testigo no cumple la ecuación
    matricial')
190
191 # Calculamos la dispersión del ruido para corregir el efecto
    del token de actualización sobre las firmas. Lo calculamos
192 # con el token en reales por seguridad
193 token_float = token.astype(float)
194 self.SIGMA_upd = (self.s_params[self.epoc_param]**2 * self.I_m)
    - (self.s_params[self.epoc_param - 1]**2 * (token_float @
    token_float.T))
195 self.SIGMA_upd = (self.SIGMA_upd + self.SIGMA_upd.T) / 2
196
197 # Comprobamos que la matriz sea semidefinida positiva. Para
    ello tomamos el parámetro gaussiano de esta etapa y la
    anterior
198 s_new = self.s_params[self.epoc_param]
199 s_old = self.s_params[self.epoc_param - 1]
200
201 # Calculamos el máximo autovalor del producto del token por su
    traspuesta
202 token_norm = np.linalg.norm(token.astype(float), 2)
203

```

```

204 # Comprobamos que sea semidefinida positiva
205 if (s_new - s_old * token_norm <= 0):
206     raise ValueError('La matriz de covarianza de la
        actualización no es semidefinida positiva')
207
208 # Si lo es, entonces calculamos el ruido gaussiano
209 r_noise_real = self.rng.multivariate_normal(
210     mean=np.zeros(self.m_param,),
211     cov=(self.r_param**2) * self.SIGMA_upd
212 )
213
214 r_noise = np rint(r_noise_real).astype(np.int64)
215
216 # Tomamos la firma antigua
217 u_e_old = sign[0]
218
219 # Calculamos la nueva firma actualizada
220 u_e = (token @ u_e_old) + r_noise
221
222 # Calculamos el nuevo testigo
223 witness_e = (token @ witness_e_old) + r_noise
224
225
226 # Comprobamos que la nueva firma sigue estando acotada
227 if np.linalg.norm(u_e) > self.beta_params[self.epoc_param]:
228     raise ValueError('La firma actualizada supera la cota')
229
230 # Comprobamos que el nuevo testigo también sigue acotado
231 if np.linalg.norm(witness_e) >
    self.beta_params[self.epoc_param]:
232     raise ValueError('El nuevo testigo supera la cota')
233
234 # Calculamos la nueva afirmación pública asociada al nuevo
    testigo
235 statement_e = (self.pk @ witness_e) % self.q_param
236
237 # Ciframos el testigo
238 upd_sign = (u_e, statement_e,
    VE_impostor_encrypt(witness_e, self.VE_pk))
239
240 # Tomamos la clave pública anterior
241 _, pk_old = self.public_keys[self.epoc_param - 1]
242
243 # Comprobamos si se cumple la ecuación con el mismo síndrome
    tras la actualización de firma
244 if not np.array_equal(((self.pk @ u_e) - statement_e) %

```

```

        self.q_param,
245         ((pk_old @ u_e_old) - sign[1]) % self.q_param):
246         raise ValueError('La actualización no preserva la relación
            entre las claves y la firma')
247
248     # Registramos la firma actualizada
249     self.reg_sign(upd_sign)
250
251     return upd_sign

```

Listing 3.8: Definición de KeyGen, Sig, Ver, Next y Update

Para ejecutar el esquema, recomendamos este flujo de ejecución para evitar errores. Dado que es una prueba de concepto, no se han implementado funciones de rollback ni control flujo, por lo que siguiendo esta estructura garantizamos su funcionalidad adecuada. Cada bloque que presentamos a continuación es inseparable, pero se pueden combinar los bloques a voluntad.

```

1  if __name__ == '__main__':
2
3      inicio = datetime.now()
4
5      # Este bloque como ejecución inicial para iniciar el sistema
6      system = usgpv(32, 5, 22900)
7      system.setup()
8
9      # Este bloque para generar la primera época
10     pk0, sk0 = system.KeyGen()
11     vepk0 = system.VE_impostor_KeyGen()
12
13     # Este bloque para firmar un mensaje y verificar si es válido
14     m = "mensaje de prueba"
15     sign = system.Sig(m)
16     print(system.Ver(vepk0, pk0, m, sign))
17
18     # Y este bloque para cambiar de época y actualizar la firma
19     pk1, sk1, token = system.Next(pk0)
20     sign1 = system.Update(token, sign)
21     print(system.Ver(system.VE_pk, pk1, m, sign1))
22
23     fin = datetime.now()
24
25     print(f'Tiempo de ejecución total: {fin - inicio}')

```

Listing 3.9: Ejemplo de ejecución

Capítulo 4

Evaluación y análisis de resultados

En este capítulo final se evalúa el rendimiento del esquema propuesto. En particular, se ajustan sus parámetros con el fin de verificar las condiciones de corrección y seguridad, se compara su comportamiento con el de la versión predecesora y se analiza la calidad de la distribución de las firmas generadas.

4.1. Ajuste de parámetros

4.1.1. Motivación

En primer lugar, es conveniente justificar la importancia de este apartado. Si definimos todos los parámetros como se especifica en [8] más lo que hemos definido durante la implementación, vamos a ver que el esquema es deficiente en términos de seguridad.

A la hora de ajustar parámetros, no podemos reducir $p_0 = P[R_{ij} \neq 0]$ demasiado por motivos de seguridad. Recordemos que, salvo el valor de las entradas de R , el adversario conoce A y toda su estructura, incluyendo las dimensiones de R . Entonces el adversario puede tomar $A = [\bar{A}|G - \bar{A}R]$, define $A_2 = G - \bar{A}R$, $M = G - A_2$ y plantea el sistema

$$\bar{A}r^{(i)} \equiv m^{(i)} \pmod{q}, \quad \forall i = 1, \dots, w$$

con $r^{(i)}$ y $m^{(i)}$ las i -ésimas columnas de R y M respectivamente. Teniendo en cuenta que por columna hay aproximadamente $p_0\bar{m}$ entradas no nulas, el problema se reduce a probar combinaciones de entradas no nulas para cada columna de R que den soluciones al sistema de ecuaciones anterior. En un vector de tamaño $\bar{m}$ las diferentes formas de colocar $p_0\bar{m}$ coordenadas no nulas son:

$$N_{col} = \binom{\bar{m}}{[p_0\bar{m}]} \cdot 2^{[p_0\bar{m}]}$$

que puede ser asumible si p_0 es demasiado pequeño. Por ejemplo, hemos elegido la distribución $P = [0,02, 0,96, 0,02]$ en la implementación. Para este caso, aproximadamente un 4% de las entradas de cada columna de R son no nulas, por lo que $[p_0\bar{m}] = [0,04 \cdot 224] = 9$.

Entonces nos saldrían alrededor de

$$N_{tot} = 224 \binom{224}{9} \cdot 2^9 \approx 3,8 \times 10^{20} \approx 2^{68}$$

combinaciones posibles, teniendo en cuenta que lo repetimos por cada una de las w columnas de R . Por tanto, esto sitúa el coste de este ataque en torno a 2^{68} operaciones, muy por debajo de los 2^{128} intentos asociados a un nivel de seguridad de 128 bits.

Adicionalmente, nos interesa conocer una aproximación del valor de $\|R\|_2$, ya que de el depende (entre otros) el valor de la segunda condición 3.2. Vamos a presentar un resultado [7, Teorema 2.1] que nos va a permitir estimar dicho valor.

Teorema 4.1 (Convergencia de los valores singulares extremos). *Sea $A = A_{n,m}$ una matriz aleatoria de tamaño $n \times m$ cuyas entradas son copias independientes de una variable aleatoria con media cero, varianza unitaria y cuarto momento finito. Supongamos que $n, m \rightarrow \infty$ y que*

$$\frac{m}{n} \rightarrow y \in (0, 1].$$

Entonces,

$$\frac{1}{\sqrt{n}} s_{\min}(A) \rightarrow 1 - \sqrt{y}, \quad \frac{1}{\sqrt{n}} s_{\max}(A) \rightarrow 1 + \sqrt{y}$$

casí seguramente.

A partir de este resultado, dado que $\|A\|_2 = s_{\max}(A)$ podemos afirmar que

$$\|A\|_2 \sim \sqrt{n} + \sqrt{m}$$

En nuestro caso, las entradas R_{ij} tienen media cero y varianza p_0 por lo que directamente no podemos aplicar el resultado. Podemos definir una variable Z normalizada tal que

$$Z = \frac{R}{\sqrt{p_0}}.$$

Entonces las entradas de Z tienen media cero, varianza unitaria y cuarto momento

$$\mathbb{E}[Z^4] = \frac{1}{p_0^2} \mathbb{E}[R^4] = \frac{1}{p_0^2} (1^4 \cdot \frac{p_0}{2} + (-1)^4 \cdot \frac{p_0}{2} + 0 \cdot (1 - p_0)) = \frac{1}{p_0}$$

que es finito dado que $p_0 > 0$. Aplicando el teorema anterior a $Z \in \mathbb{R}^{\bar{m} \times w}$, obtenemos

$$\|Z\|_2 \sim \sqrt{\bar{m}} + \sqrt{w}.$$

Como $R = \sqrt{p_0} Z$, se sigue que

$$\|R\|_2 = \sqrt{p_0} \|Z\|_2 \sim \sqrt{p_0} (\sqrt{\bar{m}} + \sqrt{w}).$$

A continuación, proporcionamos un pequeño script para estimar fácilmente el valor de $\|R\|_2$ para un p_0 cualquiera, comprobando que coincide con lo que predice el teorema 4.1


```

1 def estimador_norma_R():
2
3     size = 1000
4     p0 = 0.04
5
6     R_list = []
7     rng = np.random.default_rng()
8
9     for i in range(size):
10         R = rng.choice([-1, 0, 1], size=(224, 224), p=[p0/2, 1-p0,
11             p0/2]).astype(np.int64)
12         R_list.append(np.linalg.norm(R.astype(float), 2))
13
14     R_list = np.asarray(R_list)
15     print('='*40)
16     print('')
17     print(f'Tamaño de la muestra: {size}')
18     print(f'Probabilidad de entrada no nula: {p0}')
19     print(f'Media de la norma espectral de la muestra:
20         {np.mean(R_list)}')
21     print('')
22     print('='*40)

```

Listing 4.1: Código para estimar la norma espectral de R sobre una muestra de tamaño 1000 y $p_0 = 0.04$

A modo de resumen, en la siguiente tabla podemos ver como evoluciona el nivel de dificultad de averiguar R a medida que aumenta la estimación de su norma espectral.

p_0	$p_0 \bar{m}$	$k = \lceil p_0 \bar{m} \rceil$	$N_{tot} = w\binom{\bar{m}}{k} 2^k$	$\log_2(N_{tot})$	$\sim \ R\ _2$
0,02	4,48	5	$3,22 \times 10^{13}$	44,9	4,23
0,03	6,72	7	$1,46 \times 10^{17}$	57,0	5,18
0,04	8,96	9	$3,81 \times 10^{20}$	68,4	5,99
0,05	11,20	12	$2,27 \times 10^{25}$	84,2	6,69
0,10	22,40	23	$2,57 \times 10^{40}$	134,2	9,47
0,15	33,60	34	$7,54 \times 10^{52}$	175,7	11,59
0,20	44,80	45	$3,30 \times 10^{63}$	211,0	13,39
0,25	56,00	56	$5,03 \times 10^{72}$	241,5	14,97
0,30	67,20	68	$2,00 \times 10^{81}$	270,1	16,40

Tabla 4.1: Estimaciones del coste de búsqueda para recuperar R y del valor de la norma espectral en función de p_0 .

Concluimos entonces que no vale cualquier elección arbitraria de parámetros, sino que hay que tener en cuenta las condiciones de correctitud y de seguridad. Cabe aclarar que esto no es un análisis completo del nivel de seguridad, sino una condición mínima sobre un tipo de ataque por fuerza bruta sobre la matriz R .

4.1.2. Cálculo de parámetros

Una vez explicada la motivación, pasemos con el ajuste en si. Aclaremos que los parámetros que aportamos a continuación sirven para una prueba controlada del esquema, pero no garantizan la seguridad en una versión desplegada públicamente.

Fijémonos en que una elección óptima desde el punto de vista de la seguridad sería $p_0 \in [0,25,0,3]$. De esta forma, el coste del ataque por fuerza bruta sobre la trapdoor se situaría cerca de los 2^{256} intentos, asociados a un nivel de seguridad de 256 bits. Elegiremos $p_0 = 0,3$.

Por otro lado, dado que $\tilde{S}$ tiene una estructura fija, podemos calcular $\|\tilde{S}\| = \sqrt{5}$. Si combinamos 3.1 y 3.2, obtenemos el intervalo

$$L_{inf} = C_1 \frac{5 \log_2 w}{r^2} \leq \sigma_G \leq \frac{s_e^2}{\|R\|_2^2 + 1} - 2 = L_{sup} \quad \forall e = 1, \dots, e_{max}$$

En particular, se debe cumplir para mín $s_e = s_0$, por lo que aproximando $\|R\|_2$ por la estimación calculada, obtenemos aproximadamente que

$$\sigma_G \leq \frac{1,6^2 \cdot 224}{(16,4)^2 + 1} - 2 = 0,124$$

Ahora, notemos que aunque en [8] se indica que $r = \omega(\sqrt{\log n})$, en nuestro caso debe ser $r = \omega(\sqrt{\log_2 w})$. Esto se debe a que el paper asume que el muestreo se hace sobre el retículo definido por A , pero en esta implementación, como no disponemos en general de una base de dicho retículo, reducimos el problema sobre el retículo definido por G del cual conocemos la base S . Por lo tanto, el parámetro relevante en el muestreo de Klein (que es donde se impone la condición de suavidad) indica que $r = \omega(\sqrt{\log_2 w})$. Si nos fijamos en la primera desigualdad, vemos como la idea es que r domine al numerador. En consecuencia, definimos $r = \lceil \log_2 w \rceil$ y la constante $C_1 = 1,5$ para tener holgura suficiente. La desigualdad nos quedaría

$$1,6 = 1,5 \cdot \frac{5 \log_2 224}{8^2} \leq \sigma_G \leq 0,124$$

que es imposible de cumplir. El objetivo entonces es aumentar s_0 para subir L_{sup} . Como $\|R\|_2 \sim 2\sqrt{p_0 w}$, necesitamos que $s_0 = \omega(\sqrt{p_0 w})$. Tomaremos entonces $s_0 = p_0 w$ dando como resultado el siguiente intervalo

$$1,6 \leq \sigma_G \leq 14,73$$

Esta modificación nos obliga a ajustar β_0 ya que debe ser ligeramente superior al tamaño típico de la firma, por lo que en virtud del teorema 4.1, tomamos $\beta_0 = C_1 r s_0 \sqrt{m}$

Adicionalmente, notemos que en general se escoge $s_0 \gg 1$, luego $s_e/(s_e + 1) \approx 1$. Si nos fijamos en la tercera condición 3.3, usando lo anterior y el resultado 4.1 podemos hacer la siguiente simplificación

$$c_{upd} \geq \|\Delta_{e+1}\|_2 \sim rs_e(\sqrt{m} + \sqrt{m}) = 2rs_e\sqrt{m}$$

Ahora nos falta determinar c_{upd} . Notemos que si utilizásemos el parámetro de dispersión de la época e para generar el token, el crecimiento sería aún mayor puesto que la norma está escalada por dicho parámetro. No obstante, dado que el token no tiene que verificar ninguna cota β_i , podemos generarlo siempre con dispersión rs_0 , de modo que c_{upd} sería constante con una cota inferior aproximada

$$c_{upd} \geq 2rp_0w\sqrt{m} \approx 22860$$

Como vemos, es una constante de actualización enorme que dispara tanto los parámetros de dispersión como las cotas para cada época.

```

--- Gaussian parameters s_i ---
s_required_1 = 9.371912444808052
s_required_2 = 55.616498927066914
s_1 = 67.2
s_2 = 1514040.0
s_3 = 33611710200.0
s_4 = 746179966462200.0
s_5 = 1.6565195255460862e+19

--- Signature bounds beta_i ---
beta_1 = 4121.6
beta_2 = 91969405.43
beta_3 = 2041721270431.43
beta_4 = 4.532621220404763e+16
beta_5 = 1.006241910929858e+21

=====

```

Figura 4.1: Evolución de s_e y β_e para $e = 1, \dots, 5$

Por lo tanto, el esquema presenta un serio problema a la hora de aumentar el número de actualizaciones permitidas. Si calculamos el crecimiento por época, sería

$$\begin{aligned}
s_0 &= p_0w, \\
s_1 &= c_{upd}(s_0 + 1) = c_{upd}(p_0w + 1), \\
s_2 &= c_{upd}(s_1 + 1) = c_{upd}^2 p_0w + c_{upd}^2 + c_{upd}, \\
s_3 &= c_{upd}(s_2 + 1) = c_{upd}^3 p_0w + c_{upd}^3 + c_{upd}^2 + c_{upd}
\end{aligned}$$

que generalizando nos da la siguiente expresión

$$s_e = c_{upd}^e p_0w + \sum_{i=1}^e c_{upd}^i = c_{upd}^e p_0w + c_{upd} \frac{c_{upd}^e - 1}{c_{upd} - 1}$$

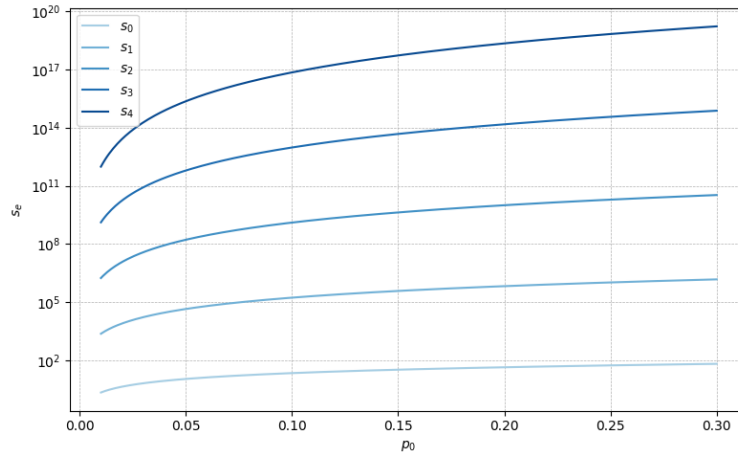


Figura 4.2: Crecimiento de s_e por épocas respecto p_0 en escala logarítmica.

Si tomamos $c_{upd} = 2p_0w\sqrt{m}\lceil\log_2(w)\rceil$ entonces podemos ver el crecimiento del parámetro en función de p_0 .

Se puede ver que en cualquier época, un aumento de $p_0 = 0,05$ a $p_0 = 0,1$ eleva un par de ordenes de magnitud el parámetro de dispersión. Por otro lado, si fijamos $p_0 = 0,3$ podemos ver que el crecimiento entre épocas es significativo, siendo por ejemplo un crecimiento de 3 órdenes de magnitud entre las épocas $e = 1$ y $e = 2$.

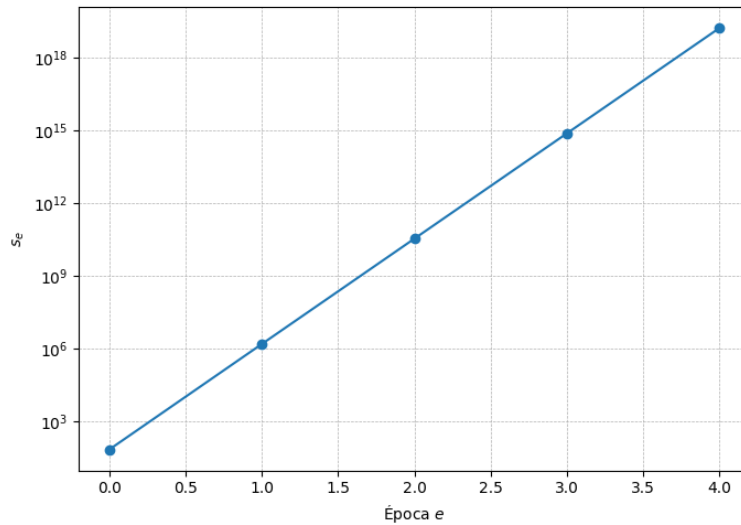


Figura 4.3: Crecimiento de s_e por épocas con $p_0 = 0,3$ en escala logarítmica.

No obstante, lo realmente interesante es hacer el ejercicio de comparar la cota de firma β

entre los esquemas CRSST21 y USGPV. Tras N actualizaciones tendríamos que

$$\frac{\beta_N^{USGPV}}{\beta_N^{CRSST21}} = \frac{c_{upd}^N C_1 r p_0 w \sqrt{m} + c_{upd} \sqrt{m} \frac{c_{upd}^N - 1}{c_{upd} - 1}}{c_{upd}^N C_1 r p_0 w \sqrt{m}} \approx 1 + \frac{1}{C_1 r p_0 w} \approx 1,0012$$

siempre que c_{upd} sea tan grande como hemos definido. Puesto que el crecimiento de β^{USGPV} es aproximadamente un 0,12 % superior al de $\beta^{CRSST21}$, concluimos que el crecimiento de β en ambos esquemas similar, lo que indica que el nuevo esquema no agrava el crecimiento acelerado de la cota de firma.

4.2. Comparación de rendimiento entre esquemas

Vamos a comparar el rendimiento de ambos esquemas sobre una tarea simulada. La implementación de CRSST21 es una adaptación del código presentado, eliminando los pasos que añade USGPV y adaptando algunos parámetros. Los parámetros utilizados en la simulación serán los siguientes:

Parámetro	USGPV	CRSST21
Semilla	seed = 2026	seed = 2026
Dimensión de salida	$n = 32$	$n = 32$
Número de épocas	$N = 5$	$N = 5$
Módulo	$q = 2^{\lceil 1,3 \log_2(n) \rceil} = 128$	$q = 2^{\lceil 1,3 \log_2(n) \rceil} = 128$
Número de bits	$k = \log_2(q) = 7$	$k = \log_2(q) = 7$
Dimensión gadget	$w = nk = 224$	$w = nk = 224$
Dimensión de entrada	$m = 2n \log_2(q) = 448$	$m = 2n \log_2(q) = 448$
Probabilidad de entrada no nula	$p_0 = 0,3$	$p_0 = 0,3$
Parámetro de escala del sampler	$r = \lceil \log_2(w) \rceil = 8$	$r = \lceil \log_2(w) \rceil = 8$
Parámetro gadget	$\sigma_G = 10$	$\sigma_G = 10$
Parámetro inicial de dispersión	$s_0 = p_0 w = 67,2$	$s = p_0 w = 67,2$
Parámetro de dispersión por época	$s_{e+1} = c_{upd} s_e + c_{upd}$	s fijo para todas las épocas
Estimación de c_{upd}	$c_{upd} = 22900$	$c_{upd} = 22900$
Cota inicial de firma	$\beta_0 = C_1 r s_0 \sqrt{m} = 4121$	$\beta_0 = C_1 r s \sqrt{m} = 4121$
Margen de aceptación	$C_1 = 1,5$	$C_1 = 1,5$
Cota de firma por época	$\beta_{e+1} = c_{upd} \beta_e + c_{upd} \sqrt{m}$	$\beta_{e+1} = c_{upd} \beta_e$

Tabla 4.2: Comparación de los parámetros utilizados en USGPV y CRSST21 para la simulación.

La simulación se divide en tres épocas: primero generamos 100 firmas, en la segunda época actualizamos esas 100 firmas y en la última volvemos a actualizar las 100 firmas y generamos 100 más. Para que la comparación sea justa, utilizaremos la semilla 2026 en ambos casos y, para que sea clara, las firmas se generarán de una en una, de forma que podremos ver el efecto acumulado de las diferencias entre cada esquema. Además, repetiremos la simulación 10 veces y lo que se representará en la siguiente tabla es la media de esas 10 simulaciones. En el apéndice se encuentra el código para correr una simulación completa. La simulación se llevará a cabo haciendo uso de la función `simulacion` y la monitorizaremos con la función `medir_rendimiento`, ambas incluidas en el anexo.

Métrica	USGPV	CRSST21	Diferencia (%)
Uso medio CPU (%)	29.06	25.97	+11.9
Uso medio CPU época 0 (%)	24.37	25.01	-2.6
Uso medio CPU época 1 (%)	34.95	75.15	-53.5
Uso medio CPU época 2 (%)	30.62	23.69	+29.3
Pico máx. de memoria total (MB)	40.58	36.01	+12.7
Pico máx. de memoria en época 0 (MB)	28.88	27.57	+4.8
Pico máx. de memoria en época 1 (MB)	21.88	14.81	+47.7
Pico máx. de memoria en época 2 (MB)	19.61	19.80	-1.0
Tiempo total de ejecución (s)	71.54	43.11	+65.9
Tiempo de ejecución en época 0 (s)	25.11	20.31	+23.6
Tiempo de ejecución en época 1 (s)	10.95	1.39	+687.8
Tiempo de ejecución en época 2 (s)	35.44	21.38	+65.8

Tabla 4.3: Comparación del uso de recursos y tiempos de ejecución entre USGPV y CRSST21.

La diferencia porcentual se ha calculado de la siguiente forma

$$\frac{\text{USGPV} - \text{CRSST21}}{\text{CRSST21}} \times 100$$

Como podemos observar, USGPV introduce un sobrecoste claro respecto a CRSST21, especialmente en tiempo de ejecución. El mayor incremento se observa en la época 1, donde se evalúa exclusivamente el proceso de actualización.

Como es sabido, la principal diferencia entre un esquema y otro es el proceso de actualización, que es lo que se evalúa exclusivamente en dicha época. Por eso, parece razonable que el tiempo de ejecución se dispare, pasando de una media de 0.01 segundos por firma actualizada a una media de 0.10 segundos. A su vez, el uso de memoria RAM se incrementa en un 47.7 %. Sin embargo, pareciera que el uso de CPU disminuye pero la realidad es diferente. El porcentaje de uso de la CPU refleja que fracción de los segundos del proceso ocupa el procesador. Para ver el tiempo de uso del procesador en segundos, hay que multiplicarlo por el tiempo de ejecución de la etapa. En ese caso, obtenemos que

Métrica	USGPV	CRSST21	Diferencia (%)
Uso medio CPU época 1 (%)	34.95	75.15	-53.5
Tiempo de ejecución en época 1 (s)	10.95	1.39	+687.8
Tiempo efectivo de CPU (s)	3.82	1.04	+267.3

Tabla 4.4: Análisis de la época 1 incluyendo tiempo de uso efectivo de CPU.

4.3. Análisis estadístico

Finalmente, nos falta analizar varios aspectos estadísticos del nuevo esquema. En primer lugar, queremos comprobar cómo de cerca está la implementación de muestrear las firmas con la dispersión esperada. Y en segundo lugar, vamos a analizar si la técnica de convolución aplicada cumple con su función y redondea la distribución de un conjunto de firmas. Como prueba de contraste, haremos lo mismo con una muestra obtenida del algoritmo CRSST21.

Por tanto, vamos a generar una muestra de firmas de tamaño N , después calcularemos la covarianza de la muestra y la compararemos con la covarianza esperada. Dicho esto, los valores que vamos a comparar son: norma de Frobenius, norma espectral, traza, máximo y mínimo autovalor. Lo haremos con la función `analisis_estadistico` que se encuentra en el anexo.

Métrica	$N = 1,000$	$N = 10,000$	$N = 50,000$	$N = 100,000$
Norma euclídea de la media muestral	375	114	50	35
Error de Frobenius relativo (%)	67	21	9	7
Error espectral relativo (%)	176	47	20	14
Error traza relativo (%)	0.02	0.12	0.04	0.01
Error máx autovalor relativo (%)	176	47	20	14
Error mín autovalor relativo (%)	89	38	18	13

Tabla 4.5: Análisis del error entre covarianza esperada y calculada por tamaño de la muestra.

Observamos que el error de todas las métricas decae con el tamaño de N , dado que la estimación mejora. La traza converge rápidamente ya que es la suma de las varianzas de cada variable, por lo que al sumar se cancelan posibles errores de estimación, aproximándose rápidamente al valor esperado. A su vez, la media muestral es bastante reducida en relación con la dispersión total de la muestra, por lo que podemos afirmar que los datos apuntan a una distribución alineada con la esperada.

Por último, vamos a analizar si la técnica de ocultación de la acción del token funciona. Para ello, realizaremos un análisis en componentes principales sobre dos muestras: una generada con CRSST21 y otra con USGPV. El análisis se ha realizado haciendo uso de la función `analisis_estadistico_pca` que se encuentra en el anexo. Hemos representado las primeras 50 componentes principales para que la visión sea suficientemente amplia. Además, la muestra usada en ambos casos es el resultado de firmar 5000 palabras aleatorias con cada esquema

y actualizarlos 3 veces, dado que nos topamos con problemas de overflow al actualizar más veces. Por ejemplo, al actualizar 4 veces, el porcentaje de varianza explicada por las primeras componentes se dispara y se iguala con los porcentajes de CRSST21, lo cual vuelve a reforzar la idea de que el ritmo de crecimiento de la dispersión de la muestra, es demasiado rápido.

En la imagen 4.4, podemos ver evidencia a favor de la hipótesis de corrección del ruido gaussiano: en USGPV la varianza aparece repartida más uniformemente entre las componentes principales que en CRSST21, donde se concentra notablemente en las primeras componentes.

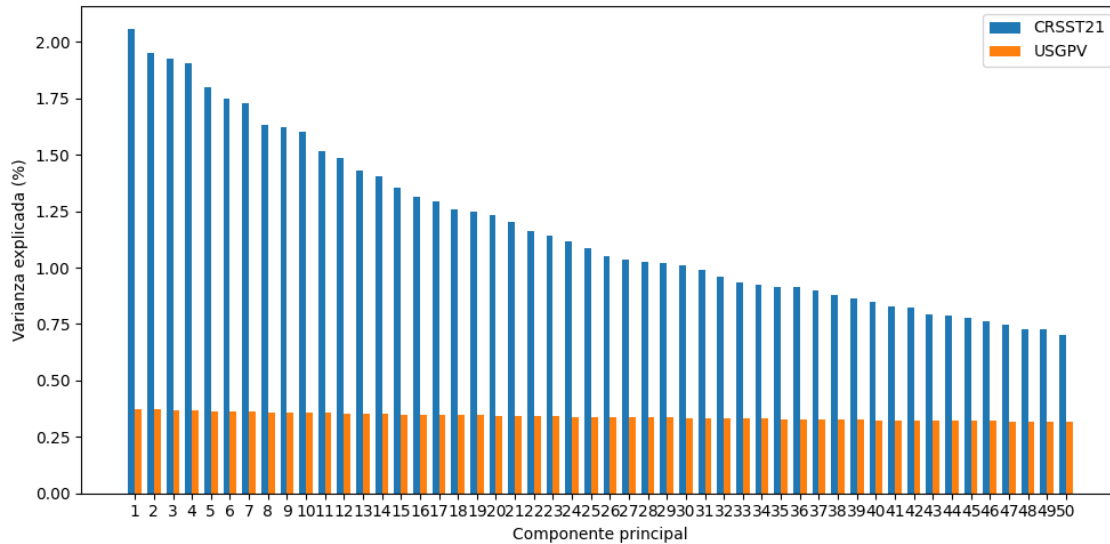


Figura 4.4: Análisis PCA sobre dos muestras de tamaño 5000 generadas con CRSST21 y USGPV.

Como podemos observar, en las 5 primeras componentes principales, USGPV produce una reducción del porcentaje de varianza explicada de más del 70 %. Más aún, la diferencia del porcentaje de varianza explicada entre la primera componente principal y la quincuagésima, es menor de 0.1, lo que indica un reparto muy uniforme de la varianza. Por tanto, el resultado obtenido aporta evidencia experimental favorable a la hipótesis, dentro de los parámetros y número de actualizaciones considerados.

Conclusión

Para concluir, podemos hacer un repaso del camino seguido durante el desarrollo del trabajo. Tras introducir los conceptos matemáticos necesarios sobre retículos y algunas definiciones fundamentales de criptografía, se vio cómo uno de los principales problemas del marco de firma CRSST21 era su proceso de actualización determinista. Este diseño permitía vincular firmas entre etapas y generaba riesgos relacionados con la posible recuperación del token de actualización por parte de un adversario.

A partir de este punto, se describió el concepto de convolución gaussiana y cómo podía aprovecharse no solo para probabilizar el proceso de actualización, sino también para ocultar la huella estadística del efecto del token sobre la distribución de las firmas. Este efecto es especialmente relevante porque se acumula con cada cambio de época. La explicación vino acompañada de una descripción detallada del nuevo método propuesto en el artículo revisado.

Posteriormente, se llevó a cabo la implementación de una prueba de concepto evaluable en Python. Para ello, fue necesario adaptar algunas nociones del esquema teórico y describir dos algoritmos adicionales basados en implementaciones ya existentes: **TrapGen** y **SamplePre**. Además, se revisaron las condiciones necesarias para la correctitud del esquema y se incluyó el código completo comentado.

Finalmente, se desarrolló un análisis de los parámetros del esquema, se evaluó el rendimiento de CRSST21 y USGPV en una misma simulación, y se estudió la dispersión efectiva de las firmas para comprobar el efecto de la corrección mediante convolución gaussiana. De este último capítulo se extraen varias conclusiones.

En primer lugar, se observó que aumentar la densidad de entradas no nulas en la trapdoor R , aunque resulta recomendable desde el punto de vista de la seguridad, perjudica el control del crecimiento de los parámetros de dispersión y de las cotas de aceptación de firmas. No obstante, los resultados muestran que este problema no es introducido por USGPV, sino heredado de CRSST21, debido al efecto acumulado del producto por la norma del token de actualización. En particular, se comprobó que las cotas de firma de ambos esquemas presentan un crecimiento muy similar en términos porcentuales.

En segundo lugar, se comprobó que USGPV introduce aumentos significativos en el uso de CPU, memoria RAM y tiempo de ejecución. Este sobre coste era esperable, ya que el esquema incorpora operaciones adicionales, como el muestreo de ruido gaussiano y el tratamiento del testigo cifrado. Aun así, parte de este coste podría mitigarse mediante paralelización, por lo que sería necesario un análisis más específico para valorar su impacto en una implementación optimizada.

Por último, se llevó a cabo un análisis PCA sobre ambos esquemas para estudiar el porcentaje de varianza explicado por cada componente principal. Según lo esperado, USGPV repartió más uniformemente la varianza entre sus componentes principales, reduciendo el peso de las componentes dominantes respecto a CRSST21. Esto es coherente con el objetivo del esquema: enmascarar probabilísticamente la acción del token de actualización y reducir la deformación estadística producida por una actualización determinista.

Por tanto, se concluye que USGPV parece mitigar con bastante eficacia el problema de la actualización determinista de CRSST21, al camuflar mejor el efecto del token sobre la distribución de las firmas, sin aumentar el ritmo de crecimiento de la dispersión y de las cotas de aceptación respecto al esquema original. No obstante, insistimos nuevamente en que esta observación debe interpretarse como evidencia experimental del prototipo, no como una demostración de seguridad de la implementación.

En contraste, sí introduce un coste computacional considerable y, sobre todo, no resuelve el principal problema estructural heredado de CRSST21: el crecimiento elevado de los parámetros de dispersión y de las cotas de firma. Este crecimiento provoca que, tras pocas actualizaciones, aparezcan problemas numéricos en la implementación, lo que limita claramente la escalabilidad práctica del prototipo desarrollado.

Como trabajo futuro, sería necesario cambiar un esquema de cifrado verificable impostor (VE) por una implementación real, emplear un sampler gaussiano discreto criptográficamente seguro, estudiar parámetros compatibles con niveles de seguridad estándar y optimizar el coste computacional mediante paralelización.

Bibliografia

- [1] Valerio Cini, Sebastian Ramacher, Daniel Slamanig, Christoph Striecks, and Erkan Tairi. Updatable signatures and message authentication codes. In Juan A. Garay, editor, *Public-Key Cryptography – PKC 2021*, pages 691–723, Cham, 2021. Springer International Publishing. URL: https://link.springer.com/chapter/10.1007/978-3-030-75245-3_25#Sec7.
- [2] Ramsès Fernàndez-València. Isogènies, codis i reticles en criptografia postquàntica. *Butlletí de la Societat Catalana de Matemàtiques*, 34(2):125–151, 2019. URL: <https://raco.cat/index.php/ButlletisCM/article/view/369473>.
- [3] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. Cryptology ePrint Archive, Paper 2007/432, 2008. <https://eprint.iacr.org/2007/432>.
- [4] Jiwon Lee, Jaekyoung Choi, Jihye Kim, and Hyunok Oh. SAVER: SNARK-compatible verifiable encryption. In *Financial Cryptography and Data Security*, Lecture Notes in Computer Science, pages 209–226. Springer, 2024. URL: <https://fc24.ifca.ai/preproceedings/61.pdf>, doi:10.1007/978-3-031-78679-2_11.
- [5] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. Cryptology ePrint Archive, Paper 2011/501, 2011. URL: <https://eprint.iacr.org/2011/501>.
- [6] Chris Peikert. An efficient and parallel gaussian sampler for lattices. Cryptology ePrint Archive, Paper 2010/088, 2010. URL: <https://eprint.iacr.org/2010/088>.
- [7] Mark Rudelson and Roman Vershynin. Non-asymptotic theory of random matrices: extreme singular values. In *Proceedings of the International Congress of Mathematicians*, volume III, pages 1576–1602, New Delhi, 2010. Hindustan Book Agency.
- [8] Haotian Yin, Jie Zhang, Wanxin Li, Yuji Dong, Eng Gee Lim, and Dominik Wojtczak. Updatable signature from lattices. Cryptology ePrint Archive, Paper 2025/1703, 2025. URL: <https://eprint.iacr.org/2025/1703>.

Apéndice

A Esquema VE impostor

```
1 import io
2 import numpy as np
3 from nacl.public import PrivateKey, SealedBox
4
5
6 def VE_keygen():
7     '''
8     Genera par de claves asimétricas.
9     '''
10    sk = PrivateKey.generate()
11    pk = sk.public_key
12    return pk, sk
13
14
15 def vector_to_bytes(v: np.ndarray) -> bytes:
16     '''
17     Serializa vector/matriz NumPy sin pickle.
18     '''
19    buffer = io.BytesIO()
20    np.save(buffer, v, allow_pickle=False)
21    return buffer.getvalue()
22
23
24 def bytes_to_vector(data: bytes) -> np.ndarray:
25     '''
26     Reconstruye vector/matriz NumPy.
27     '''
28    buffer = io.BytesIO(data)
29    return np.load(buffer, allow_pickle=False)
30
31
32 def VE_impostor_encrypt(v: np.ndarray, pk) -> bytes:
33     '''
```

```

34     Cifra un vector/matriz usando clave pública.
35     '''
36     plaintext = vector_to_bytes(v)
37     box = SealedBox(pk)
38     ciphertext = box.encrypt(plaintext)
39     return ciphertext
40
41
42
43
44 def VE_impostor_decrypt(ciphertext: bytes, sk) -> np.ndarray:
45     '''
46     Descifra usando clave privada.
47     '''
48     box = SealedBox(sk)
49     plaintext = box.decrypt(ciphertext)
50     return bytes_to_vector(plaintext)

```

Listing 2: Archivo VE_impostor_scheme.py

B Simulador y monitorización

```

1 def medir_rendimiento(func, *args, intervalo=0.01, **kwargs):
2     # Identificamos el proceso de la función
3     process = psutil.Process(os.getpid())
4
5     # CPU inicial del proceso
6     cpu_inicio = process.cpu_times()
7     cpu_inicio_total = cpu_inicio.user + cpu_inicio.system
8
9     # Memoria inicial del proceso
10    memoria_inicio = process.memory_info().rss / 1024 / 1024 # MB
11
12    # Creamos la lista para guardar las mediciones de memoria
13    memorias = []
14    ejecutando = True
15
16    # Esta función realizará mediciones en segundo plano
17    def monitor_memoria():
18        while ejecutando:
19            memoria_actual = process.memory_info().rss / 1024 / 1024
20            memorias.append(memoria_actual)
21            time.sleep(intervalo)
22

```

```

23 # Iniciamos la función anterior
24 hilo = threading.Thread(target=monitor_memoria)
25 hilo.start()
26
27 # Con esto guardamos el inicio de la ejecución de la función
28 t0 = time.perf_counter()
29
30 # La ejecutamos y calculamos el tiempo de ejecución
31 try:
32     resultado = func(*args, **kwargs)
33 finally:
34     tiempo_real = time.perf_counter() - t0
35
36     # Paramos el proceso en segundo plano
37     ejecutando = False
38     hilo.join()
39
40 # CPU final del proceso
41 cpu_final = process.cpu_times()
42 cpu_final_total = cpu_final.user + cpu_final.system
43
44 # Tiempo total de CPU usado por la función
45 tiempo_cpu_usado = cpu_final_total - cpu_inicio_total
46
47 # Porcentaje medio de CPU usado por la función
48 cpu_porcentaje = (tiempo_cpu_usado / tiempo_real) * 100 if
49     tiempo_real > 0 else 0
50
51 # Memoria
52 memoria_final = process.memory_info().rss / 1024 / 1024
53 memoria_pico = max(memorias) if memorias else memoria_final
54 memoria_usada = memoria_pico - memoria_inicio
55
56 print('=== Medición de la función ===')
57 print(f'Tiempo real: {tiempo_real:.6f} s')
58 print(f'CPU usado: {cpu_porcentaje:.2f} %')
59 print(f'Memoria usada: {memoria_usada:.2f} MB')
60 print(f'Memoria pico del proceso: {memoria_pico:.2f} MB')
61
62 return resultado
63
64 def generar_palabras(n=1000, longitud=10):
65
66     palabras = []
67

```

```

68     rng = random.Random(2026)
69
70     for _ in range(n):
71         palabra = ''.join(rng.choices(string.ascii_lowercase,
72                                     k=longitud))
73         palabras.append(palabra)
74
75     return palabras
76
77 def simulacion():
78     sample_size = 100
79
80     # Ahora generamos la lista de palabras que vamos a firmar
81     lista_palabras = generar_palabras(n=sample_size, longitud=10)
82
83     inicio_total = datetime.now()
84
85     # Creamos una instancia del sistema
86     print('='*50)
87     print('Iniciación del sistema')
88     system = usgpv(32, 5, 22900, seed=2026)
89
90     def firmar(sample_size, lista_palabras):
91
92         # Definimos los parámetros y generamos las claves iniciales
93         system.setup()
94         pk0, sk0 = system.KeyGen()
95         vepk0 = system.VE_impostor_KeyGen()
96
97
98         # Firmamos todas las palabras y las guardamos en una lista
99         print('='*50)
100        print(f'Firmando los {sample_size} mensajes')
101        firmas = []
102        fallos = []
103        for i, palabra in enumerate(lista_palabras):
104            # print(f'Firmando mensaje: {i}/{sample_size}')
105            sign = system.Sig(palabra)
106            if system.Ver(vepk0, pk0, palabra, sign):
107                firmas.append((palabra, sign))
108            else:
109                fallos.append((palabra, sign))
110                print(f'Error al firmar la palabra {palabra}')
111        print('='*50)
112        return firmas, fallos, pk0

```

```

113
114     print('='*50)
115     print('Estadísticas etapa 0')
116     firmas,fallos,pk0 = medir_rendimiento(firmar,sample_size,
117                                         lista_palabras)
117     print('='*50)
118     fin_epoca_0 = datetime.now()
119
120     inicio_epoca_1 = datetime.now()
121
122     def actualizacion_1(sample_size,firmas,pk0):
123         # Cambiamos una vez de época de e=0 -> e=1
124         print('='*50)
125         print('Cambio de época y actualización')
126         pk1, sk1, token = system.Next(pk0)
127
128         # Actualizamos todas las firmas
129         firmas_1 = []
130         fallos_1 = []
131         for i,firma in enumerate(firmas):
132             # print(f'Actualizando mensaje: {i}/{sample_size}')
133             sign1 = system.Update(token, firma[1])
134             if system.Ver(system.VE_pk, pk1, firma[0], sign1):
135                 firmas_1.append((firma[0],sign1))
136             else:
137                 fallos_1.append((firma[0],sign1))
138                 print(f'Error al actualizar la palabra {firma[0]}')
139
140         return firmas_1,fallos_1,pk1,token
141
142     print('='*50)
143     print('Estadísticas etapa 1')
144     firmas_1,fallos_1,pk1,token =
145         medir_rendimiento(actualizacion_1,sample_size,firmas,pk0)
146     print('='*50)
147     fin_epoca_1 = datetime.now()
148
149     inicio_epoca_2 = datetime.now()
150
151     def actualizacion_2(sample_size,firmas_1,pk1, lista_palabras):
152         # Cambiamos una vez de época de e=1 -> e=2
153         print('='*50)
154         print('Cambio de época y actualización')
155         pk2, sk2, token_2 = system.Next(pk1)
156
157         # Actualizamos todas las firmas

```



```

157     firmas_2 = []
158     fallos_2 = []
159     for i,firma in enumerate(firmas_1):
160         # print(f'Actualizando mensaje: {i}/{sample_size}')
161         sign2 = system.Update(token_2, firma[1])
162         if system.Ver(system.VE_pk, pk2, firma[0], sign2):
163             firmas_2.append((firma[0],sign2))
164         else:
165             fallos_2.append((firma[0],sign2))
166             print(f'Error al actualizar la palabra {firma[0]}')
167
168     for i,palabra in enumerate(lista_palabras):
169         # print(f'Firmando mensaje: {i}/{sample_size}')
170         sign = system.Sig(palabra)
171         if system.Ver(system.VE_pk, pk2, palabra, sign):
172             firmas_2.append((palabra,sign))
173         else:
174             fallos_2.append((palabra,sign))
175             print(f'Error al firmar la palabra {palabra}')
176     print('='*50)
177     return firmas_2,fallos_2,pk2,token_2
178
179     print('='*50)
180     print('Estadísticas etapa 2')
181     firmas_2,fallos_2,pk2,token_2 =
182         medir_rendimiento(actualizacion_2,sample_size,firmas_1,
183         pk1,lista_palabras)
184     print('='*50)
185
186     fin_total = datetime.now()
187
188     print(f'Tiempo de ejecución total: {fin_total-inicio_total}')
189     print(f'Tiempo de ejecución epoca 0:
190         {fin_epoca_0-inicio_total}')
191     print(f'Tiempo de ejecución epoca 1:
192         {fin_epoca_1-inicio_epoca_1}')
193     print(f'Tiempo de ejecución epoca 2:
194         {fin_total-inicio_epoca_2}')

```

Listing 3: Script de simulación y monitorización

C Análisis estadístico de la dispersión efectiva

```

1 def analisis_estadistico():

```

```

2  # Tamaño de la muestra
3  sample_size = 100000
4
5  # Ahora generamos la lista de palabras que vamos a firmar
6  print('='*50)
7  print(f'Generando las {sample_size} palabras')
8  lista_palabras = generar_palabras(n=sample_size,longitud=10)
9  print('='*50)
10
11 # Creamos una instancia del sistema
12 print('='*50)
13 print('Inicialización del sistema')
14 system_usgpv = usgpv(32, 5, 22900,seed=2026)
15
16
17 # Definimos los parámetros y generamos las claves iniciales
18 system_usgpv.setup()
19 pk0, sk0 = system_usgpv.KeyGen()
20 vepk0 = system_usgpv.VE_impostor_KeyGen()
21
22 # Firmamos todas las palabras y las guardamos en una lista
23 print('='*50)
24 print(f'Firmando los {sample_size} mensajes')
25 muestra_usgpv = system_usgpv.Sig(lista_palabras)
26 print('='*50)
27
28
29 # Guardamos la muestra y trasponemos para que cada columna sea
    una muestra
30 X_usgpv = np.asarray([sample[0] for sample in muestra_usgpv]).T
31
32 # Esta es la matriz usada por el sistema para muestrear
33 Sigma_esperada =
    (system_usgpv.r_param**2)*np.asarray(system_usgpv.SIGMA,
    dtype=float)
34
35 # Aquí estimamos la covarianza de la muestra usando el
    estimador insesgado
36 Sigma_empirica = np.cov(X_usgpv, ddof=1)
37
38 # Calculamos la media de la distribución, que tiene que estar
    cerca del 0
39 media_empirica = np.mean(X_usgpv,axis=1)
40 norma_media = np.linalg.norm(media_empirica)
41
42 print('='*50)

```

```

43 print(f'Norma euclidea de la media muestral: {norma_media}')
44
45 # Calculamos la norma de Frobenius de la diferencia
46 error_fro = np.linalg.norm(Sigma_empirica - Sigma_esperada,
47                             ord="fro")
48 # Calculamos la norma de Frobenius esperada
49 norma_esperada_fro = np.linalg.norm(Sigma_esperada, ord="fro")
50
51 # Calculamos el porcentaje de error relativo
52 error_relativo_fro = error_fro / norma_esperada_fro
53
54 print('='*50)
55 print(f'Error Frobenius absoluto: {np.round(error_fro,2)}')
56 print(f'Norma Frobenius esperada:
57       {np.round(norma_esperada_fro,2)}')
58 print(f'Error Frobenius relativo: {np.round(100 *
59       error_relativo_fro,2)} %')
60
61 # Calculamos la norma espectral de la diferencia
62 error_esp = np.linalg.norm(Sigma_empirica - Sigma_esperada,
63                             ord=2)
64 # Calculamos la norma espectral esperada
65 norma_esperada_esp = np.linalg.norm(Sigma_esperada, ord=2)
66
67 # Calculamos el porcentaje de error relativo
68 error_relativo_esp = np.round(error_esp / norma_esperada_esp,2)
69
70 print('='*50)
71 print(f'Error Espectral absoluto: {np.round(error_esp,2)}')
72 print(f'Norma Espectral esperada:
73       {np.round(norma_esperada_esp,2)}')
74 print(f'Error Espectral relativo: {np.round(100 *
75       error_relativo_esp,2)} %')
76
77 # Calculamos las trazas de ambas matrices
78 tr_emp = np.trace(Sigma_empirica)
79 tr_esp = np.trace(Sigma_esperada)
80
81 print('='*50)
82 print(f'Traza empírica: {np.round(tr_emp,2)}')
83 print(f'Traza esperada: {np.round(tr_esp,2)}')
84 print(f'Error relativo traza: {np.round(100 * (tr_emp - tr_esp)
85       / tr_esp,2)} %')
86
87 # Calculamos los autovalores
88 autovalores_esp = np.linalg.eigvals(Sigma_esperada)

```

```

82     autovalores_emp = np.linalg.eigvals(Sigma_empirica)
83
84     # Calculamos el error del máximo y del mínimo autovalor
85     error_auto_max_rel = 100*(autovalores_emp.max() -
86         autovalores_esp.max())/autovalores_esp.max()
87     error_auto_min_rel = 100*(autovalores_emp.min() -
88         autovalores_esp.min())/autovalores_esp.min()
89
90     print('='*50)
91     print(f'Error relativo autovalor máximo:
92         {np.round(error_auto_max_rel,2)} %')
93     print(f'Error relativo autovalor mínimo:
94         {np.round(error_auto_min_rel,2)} %')

```

Listing 4: Script de análisis de la dispersión efectiva

D Análisis estadístico PCA

```

1 def analisis_estadistico_pca():
2     sample_size = 5000
3
4     # Ahora generamos la lista de palabras que vamos a firmar
5     print('='*50)
6     print(f'Generando las {sample_size} palabras')
7     lista_palabras = generar_palabras(n=sample_size,longitud=10)
8     print('='*50)
9
10    # Creamos una instancia del sistema
11    print('='*50)
12    print('Inicialización del sistema')
13    system_usgpv = usgpv(32, 5, 22900,seed=2026)
14
15
16    # Definimos los parámetros y generamos las claves iniciales
17    system_usgpv.setup()
18    pk0, sk0 = system_usgpv.KeyGen()
19    vepk0 = system_usgpv.VE_impostor_KeyGen()
20
21    # Firmamos todas las palabras y las guardamos en una lista
22    print('='*50)
23    print(f'Firmando los {sample_size} mensajes')
24    muestra_usgpv = system_usgpv.Sig(lista_palabras)
25    print('='*50)
26

```

```

27
28 print('Primera actualizacion USGPV')
29 # Ahora vamos a actualizar la muestra para ver la acción del
    token
30 pk1, sk1, token = system_usgpv.Next(pk0)
31 muestra_actualizada_usgpv = [system_usgpv.Update(token, sign)
    for sign in muestra_usgpv]
32
33 print('Segunda actualizacion USGPV')
34 pk2, sk2, token2 = system_usgpv.Next(pk1)
35 muestra_actualizada_usgpv = [system_usgpv.Update(token2, sign)
    for sign in muestra_actualizada_usgpv]
36
37 print('Tercera actualizacion USGPV')
38 pk3, sk3, token3 = system_usgpv.Next(pk2)
39 muestra_actualizada_usgpv = [system_usgpv.Update(token3, sign)
    for sign in muestra_actualizada_usgpv]
40
41 # print('Cuarta actualizacion USGPV')
42 # pk4, sk4, token4 = system_usgpv.Next(pk3)
43 # muestra_actualizada_usgpv = [system_usgpv.Update(token4,
    sign) for sign in muestra_actualizada_usgpv]
44
45 X_usgpv = np.asarray([sample[0] for sample in
    muestra_actualizada_usgpv])
46
47 # Ahora generamos una muestra con CRSST21
48 # Creamos una instancia del sistema
49 print('='*50)
50 print('Inicialización del sistema')
51 system_crsst21 = crsst21(32, 5, 22900, seed=2026)
52
53
54 # Definimos los parámetros y generamos las claves iniciales
55 system_crsst21.setup()
56 pk0, sk0 = system_crsst21.KeyGen()
57
58 # Firmamos todas las palabras y las guardamos en una lista
59 print('='*50)
60 print(f'Firmando los {sample_size} mensajes')
61 muestra_crsst21 = system_crsst21.Sig(lista_palabras)
62 print('='*50)
63
64 print('Primera actualizacion CRSST21')
65 # Ahora vamos a actualizar la muestra para ver la acción del
    token

```

```

66 pk1, sk1, token = system_crsst21.Next(pk0)
67 muestra_actualizada_crsst21 = [system_crsst21.Update(token,
68     sign) for sign in muestra_crsst21]
69
70 print('Segunda actualizacion CRSST21')
71 pk2, sk2, token2 = system_crsst21.Next(pk1)
72 muestra_actualizada_crsst21 = [system_crsst21.Update(token2,
73     sign) for sign in muestra_actualizada_crsst21]
74
75 print('Tercera actualizacion CRSST21')
76 pk3, sk3, token3 = system_crsst21.Next(pk2)
77 muestra_actualizada_crsst21 = [system_crsst21.Update(token3,
78     sign) for sign in muestra_actualizada_crsst21]
79
80 # print('Cuarta actualizacion CRSST21')
81 # pk4, sk4, token4 = system_crsst21.Next(pk3)
82 # muestra_actualizada_crsst21 = [system_crsst21.Update(token4,
83     sign) for sign in muestra_actualizada_crsst21]
84
85 X_crsst21 = np.asarray(muestra_actualizada_crsst21)
86
87 def pca_muestra(X, n_componentes=None):
88     '''
89     Realiza un análisis PCA sobre una muestra dada.
90     Input: X (muestra), n_componentes (número de componentes
91     principales a devolver)
92     Output: componentes (número de componentes),
93     varianza_explicada (tasa de varianza explicada por
94     componentes),
95     scores (proyecciones sobre las componentes
96     principales)
97     '''
98
99     # Nos aseguramos de que la muestra es del tipo np.ndarray
100     X = np.asarray(X, dtype=float)
101
102     # En primer lugar, centramos la muestra. Aunque vimos que
103     # la media era prácticamente nula,
104     # no lo es exactamente así que es conveniente centrarla
105     # igualmente.
106     media = np.mean(X, axis=0)
107     X_centrada = X - media
108
109     # Recalculamos la matriz de covarianza empírica
110     Sigma_empirica = np.cov(X_centrada, rowvar=False, ddof=1)

```

```

102     # Para comparar las muestras, las normalizamos segun su
        dispersión total
103     # que es la raiz de la traza de la matriz de covarianza (es
        decir, la raiz
104     # de la suma de las varianzas de cada variable)
105     escala = np.sqrt(np.trace(Sigma_empirica))
106
107     # Calculamos los autovalores y autovectores
108     autovalores, autovectores = np.linalg.eigh(Sigma_empirica)
109
110     # Ordenamos de mayor a menor varianza. Esta función nos da
        los indices de los autovalores
111     # ordenados de mayor a menor
112     idx = np.argsort(autovalores)[::-1]
113
114     # Obtenemos los autovalores en ese orden
115     autovalores = autovalores[idx]
116     # Obtenemos los autovectores en ese orden
117     autovectores = autovectores[:, idx]
118
119     # Calculamos el porcentaje de varianza explicada por cada
        uno
120     varianza_explicada = autovalores / np.sum(autovalores)
121     varianza_acumulada = np.cumsum(varianza_explicada)
122
123     # nos quedamos con las primeras componentes si se pide
124     if n_componentes is not None:
125         autovalores = autovalores[:n_componentes]
126         autovectores = autovectores[:, :n_componentes]
127         varianza_explicada = varianza_explicada[:n_componentes]
128         varianza_acumulada = varianza_acumulada[:n_componentes]
129
130     # Normalizamos la muestra
131     X_norm = X_centrada / escala
132
133     # Proyectamos la muestra sobre las componentes principales
134     scores = X_norm @ autovectores
135
136     # Calculamos la lista de componentes
137     componentes = np.arange(1, len(varianza_explicada) + 1)
138
139
140     return componentes, varianza_explicada, scores
141
142     # Calculamos las componentes, la tasa de varianza explicada y
        las proyecciones sobre las componentes para CRSST21

```

```

143 print('='*50)
144 print('Calculando PCA para CRSST21')
145 componentes_crsst21, varianza_explicada_crsst21, scores_crsst21
    = pca_muestra(X_crsst21,n_componentes=50)
146 # Calculamos las componentes, la tasa de varianza explicada y
    las proyecciones sobre las componentes para USGPV
147 print('='*50)
148 print('Calculando PCA para USGPV')
149 componentes_usgpv, varianza_explicada_usgpv, scores_usgpv =
    pca_muestra(X_usgpv,n_componentes=50)
150
151 # Pasamos de decimal a porcentaje en la varianza explicada
152 var_crsst21_pca = 100 * np.asarray(varianza_explicada_crsst21)
153 var_usgpv_pca = 100 * np.asarray(varianza_explicada_usgpv)
154
155 def grafico_barras_pca(componentes, var_pca_1, var_pca_2):
156     '''
157     Representa el porcentaje de explicación de varianza de cada
        componente principal
158     ordenada de mayor a menor. Recibe dos entradas y las
        representa en paralelo.
159     Input: componentes (lista de componentes principales),
        var_pca_1 (lista de porcentajes 1)
        var_pca_2 (lista de porcentajes 2)
160     Output: None (Solo muestra la gráfica)
161     '''
162
163     ancho = 0.35
164     x = np.arange(len(componentes))
165
166     plt.figure(figsize=(10, 5))
167
168     plt.bar(
169         x - ancho/2,
170         var_pca_1,
171         width=ancho,
172         label="CRSST21"
173     )
174
175     plt.bar(
176         x + ancho/2,
177         var_pca_2,
178         width=ancho,
179         label="USGPV"
180     )
181
182     plt.xlabel("Componente principal")

```



```
183     plt.ylabel("Varianza explicada (%)")
184     plt.xticks(x, componentes)
185     plt.legend()
186     plt.tight_layout()
187     plt.show()
```

Listing 5: Script de análisis PCA